claude-windows / 7dc3da51-af38-47b6-83f6-fdda83b60664

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\subagents\workflows\wf_eb99bd9f-440\agent-a8308ed8ba75083d6.jsonl`
- SHA-256: `50f0ec9a2bf31773fa9efc09891c78c8428cc779280f2c026730ec1db2083f3e`
- Source modified: `2026-07-06T06:05:01+00:00`
- Imported at: `2026-07-08T16:00:12+00:00`
- project: `wf_eb99bd9f-440`
- session_id: `7dc3da51-af38-47b6-83f6-fdda83b60664`

Transcript

1. user (2026-07-06T06:02:53.383Z)

```
You are reviewing GitHub PR #35 of the repo at C:/Users/avidu/Projects/Telegram (branch
feat/track-b-domain-schema -> main).
It implements ADR 0011 (docs/adr/0011-provider-neutral-domain-schema.md): a provider-neutral
domain schema (Track B). Changed files ONLY: src/tg_video_filter/db.py,
src/tg_video_filter/models.py,
tests/test_db.py, tests/test_delivery.py. Diff: +1299/-150.

Ground rules:
- READ the actual code (Read/Grep). Cite exact file:line for every claim.
- You MAY run the venv python for a targeted repro:
C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe <script>. The DB layer is aiosqlite;
  use db._apply_schema(conn) on an aiosqlite ":memory:" connection to build the schema, and
  db.upsert_account(...) to seed. A working repro is stronger evidence than prose.
- Compare against v1 behaviour on main when relevant: git -C C:/Users/avidu/Projects/Telegram
show main:src/tg_video_filter/db.py
- Authoritative local gate ALREADY RUN (do not re-run the full suite): ruff check clean; pytest
  434 passed @ 96.63% coverage (floor 80%); ruff format --check FAILS only on commands.py which
is
  NOT in this PR (pre-existing on main) ? do not report that as a PR finding.
- Report ONLY defects that ship in THIS PR's diff. No style nits already passing ruff. No
praise.
- Each finding needs a concrete failure scenario (inputs -> wrong result). Rank most-severe
first.
- Severity: blocker (data loss/corruption/wrong core behaviour), high (real bug, narrow
trigger),
  medium (latent/edge), low (hygiene). Be honest; downgrade if trigger is unrealistic.

DIMENSION: test quality & coverage gaps in tests/test_db.py and tests/test_delivery.py.
Investigate specifically:
- ADR 0011 explicitly required the migration test to "seed rows hitting EACH dedup branch (incl.
  all-fingerprint-fields-null) and assert no two rows collapse to the same dedup_key". Does the
migration
  test (around test_migration_adds_document_id / the 003 assertions ~700-770) actually do that?
It appears
  to seed ONE row hitting ONE branch. Confirm the gap.
- Is the Tier-2 cross-tier regression (prior item has document_id; re-upload same fingerprint,
new doc id)
  covered by ANY test? get_video_by_content_fingerprint_hit seeds a doc-id-less video only.
- Are the shim error branches (get_source_by_id returns None; int() on None) tested? (coverage
shows
  db.py lines 1103-1104, 1147, 1172, 512 uncovered.)
- test_delivery.py changes: do they meaningfully assert the mark_forwarded no-op is safe, or
just that it
```

  doesn't crash?
Return findings per schema. A missing critical test IS a finding (severity by what bug it would have caught).

2. assistant (2026-07-06T06:02:58.339Z)

I'll review the test quality and coverage gaps. Let me start by reading the relevant files.

3. user (2026-07-06T06:02:59.332Z)

```
1       # ADR 0011: Provider-neutral domain schema (Track B)
2
3       - **Status**: Accepted (Question 1 ? Option 1A; Question 2 ? Option 2A; PR #34)
4       - **Date**: 2026-07-06
5       - **Decision owner**: project owner
6       - **Tracking issue**: #27 (Track B of the provider-neutral umbrella, #25)
7       - **Depends on**: ADR 0010 (Track A ? `accounts` + `telegram_sessions`, merged in #33)
8       - **Blocks**: #28 (Track C ? delivery/routing), #29 (Track D ? commands), and indirectly
#30 (Track E)
9
10      ## Context
11
12      ADR 0010 split `users` into `accounts` (product identity) + `telegram_sessions`
13      (MTProto credential). That unblocked the *identity* dimension. This ADR
14      tackles the **content dimension**: how to represent sources, content items,
15      policies, destinations, routes, and delivery attempts in a provider-neutral
16      way, so that the system can ingest from Telegram today and from WhatsApp
17      (or any future provider) later without another schema rewrite.
18
19      ### What we have today (post-Track-A)
20
21      | Table | Meaning | Provider-coupling |
22      |---|---|---|
23      | `accounts` | Product identity (ADR 0010) | provider-neutral (`provider` column) |
24      | `telegram_sessions` | Optional MTProto session (ADR 0010) | Telegram-specific by
design |
25      | `channels` | A Telegram channel/chat an account is a member of | **Telegram-specific**
(`channel_id` is a Telegram id) |
26      | `videos` | A seen Telegram video message + metadata | **Telegram-specific**
(`message_id`, `document_id` are Telegram ids) |
27      | `filter_config` | Per-account JSON blob of `FilterConfig` | provider-neutral in shape,
but scoped per-account not per-source |
28
29      The dedup logic (ADR 0006) is **3-tier and Telegram-specific**:
30      - Tier 0: `(user_id, channel_id, message_id)` ? same Telegram message.
31      - Tier 1: `(user_id, document_id)` ? same Telegram file forwarded across channels.
32      - Tier 2: `(user_id, size_bytes, duration_s, height, width)` ? re-upload fingerprint.
33
34      ### What we need
35
36      The umbrella (#25) introduces six concepts: `Source`, `ContentItem`, `Policy`,
37      `Destination`, `Route` (`routing_rules`), and `DeliveryAttempt`. This ADR
38      defines the schema for each, the **two open design questions** that gate the
39      schema, and the migration path from the current tables.
40
41      ## The two decisions I need from you
42
43      ### Question 1: `ContentItem.dedup_key` ? how does provider-neutral dedup relate to ADR
0006?
44
45      The current 3-tier dedup is Telegram-specific (`document_id` is a Telegram
46      concept). A provider-neutral `ContentItem` needs a dedup story that works
47      across providers. Three options:
48
49      #### Option 1A (recommended): tiered lookup that wraps ADR 0006's tiers
```

Keep ADR 0006's three tiers but generalize them behind a single
`dedup_key` column on `content_items`:

```
content_items.dedup_key TEXT NOT NULL
```

The *value* of `dedup_key` is computed by an explicit decision tree at
ingestion time, provider by provider. The tree is ordered strongest-key
first and each branch is **guarded so partial metadata can never collapse
to a degenerate key** (e.g. `tg:fp:0:0:0:0`):

- **Telegram:**
  1. If `document_id` is present ? `dedup_key = "tg:doc:{document_id}"`.
  2. Else if **all four** fingerprint fields (`size_bytes`, `duration_s`,
     `height`, `width`) are present ? `dedup_key = "tg:fp:{size}:{dur}:{h}:{w}"`.
  3. Else ? `dedup_key = "tg:msg:{source_id}:{provider_message_id}"`
     (Tier 0; the source/channel id is required because Telegram
     `message_id` is only unique within a channel, not across sources).
- **Future WhatsApp:** `dedup_key = "wa:msg:{message_id}"` (WhatsApp message
  ids are globally unique per account) or `"wa:sha256:{blob_hash}"` if WA
  exposes a content hash.

The guard in step 2 (all four fields required) is what prevents the
partial-metadata collision the first draft of this ADR allowed. The pure
`compute_dedup_key(provider, *, document_id, source_id, provider_message_id,
size_bytes, duration_s, height, width)` function that implements this tree
gets its own unit tests in the implementation PR, including cases for
partial/null metadata.

A **partial unique index** on `(account_id, dedup_key) WHERE dedup_key IS NOT NULL`
replaces the current Tier-1 index and enforces per-account dedup at the DB
level, regardless of provider.

Why recommended:
- Zero new bandwidth ? all keys are computed from metadata already extracted
  (honors ADR 0003's no-download principle).
- The provider prefix (`tg:` / `wa:`) makes cross-provider collisions
  structurally impossible.
- Existing `videos` rows migrate cleanly: `dedup_key` is computed from their
  existing `document_id` / metadata columns during the migration.
- Supersedes ADR 0006 cleanly: the three tiers become *one* column with
  provider-specific derivation rules, documented in the migration.

Costs:
- The migration must compute `dedup_key` for every existing `videos` row
  (cheap ? pure SQL string building, no I/O).
- The `dedup_key` derivation logic lives in code (a pure function per
  provider), not in the schema. This is a feature, but it means the
  derivation rules need their own unit tests.

Option 1B: content hash (SHA-256 of downloaded bytes)

`dedup_key = sha256(downloaded_content)`. Cryptographically strong,
provider-agnostic, catches re-encodes that 1A misses.

Rejected ? violates ADR 0003 (metadata-first, no download in the hot
path). Downloading every incoming video to hash it would multiply bandwidth
and latency by orders of magnitude. Could be added later as an optional
deep-inspect stage (deferred to Track F per #31).

Option 1C: provider-assigned only (drop Tier 2)

`dedup_key = "tg:doc:{document_id}"` always; no metadata fingerprint

fallback. Simpler, but loses Tier-2 dedup for re-uploads that get a new `document.id` ? a real and common case (someone re-encodes and re-uploads the same video to a different channel).

Recommendation: 1A. It preserves the dedup quality of ADR 0006, generalizes cleanly, and honors the no-download constraint.

Question 2: `Policy` ? existing `filter_config` ? rename, or wrap?

The codebase already has `filter_config.config_json` (one row per account, holding a `FilterConfig` JSON blob ? see `models.py` and ADR 0005). The umbrella introduces a `Policy` concept. They overlap. Two options:

Option 2A (recommended): `Policy` *generalizes* `filter_config` (rename + extend)

`policies` is a new table; `filter_config` is migrated into it. One account can own multiple policies (enables per-route filter overrides from ADR 0009), with a `DEFAULT policy` per account for the no-rules-configured case.

```
policies (
    id           INTEGER PRIMARY KEY AUTOINCREMENT,
    account_id   INTEGER NOT NULL REFERENCES accounts(id),
    name         TEXT NOT NULL,                    -- e.g. "default", "reggae-route"
    config_json  TEXT NOT NULL,                    -- the FilterConfig blob (Phase 1)
    is_default   INTEGER NOT NULL DEFAULT 0,       -- one per account (UNIQUE partial idx)
    created_at   TEXT NOT NULL,
    updated_at   TEXT NOT NULL,
    UNIQUE (account_id, name)
);
```

Phase 1 (this track): `config_json` holds exactly today's `FilterConfig` JSON ? the rename is purely structural. **Phase 2** (Track F, deferred): `config_json` grows to include topic preferences and security rules, but that's behind the same column so no further migration.

Migration: for each `filter_config` row, insert a `policies` row with `name='default'`, `is_default=1`, `config_json=<the existing blob>`. Drop `filter_config` after the migration (keep `filter_config_deprecated` for one release, mirroring ADR 0010's safety-net pattern).

Why recommended:
- `Policy` becomes the single home for "what to filter/enrich" ? no second overlapping concept. Today's `FilterConfig` is just the Phase-1 shape.
- Enables ADR 0009's per-route filter overrides (`routing_rules.policy_id`) without a second migration later.
- The existing `db.load_filter_config(account_id)` / `save_filter_config` become thin shims over `policies WHERE account_id=? AND is_default=1`, mirroring the Track-A `User` shim pattern ? backwards-compatible.

Costs:
- One more table migration. `filter_config` ? `policies` is a rename+extend.
- The `FilterConfig` Pydantic model stays (it's the Phase-1 config shape); `Policy` is the *container* (name, owner, default flag) around it. Two models, not one ? but they compose cleanly.

Option 2B: keep `filter_config` as-is; `Policy` is a wrapping layer

Don't touch `filter_config`. `policies` references a `filter_config` row plus adds enrichment/security stages as separate columns.

Rejected ? creates two overlapping concepts (`filter_config` for the v1 rules, `policies` for "everything else"), which is exactly the conflation the umbrella exists to dissolve. We'd be back here in a year merging them.

Recommendation: 2A. One concept, one table, phased growth via the same JSON column.

Proposed schema (assuming 1A + 2A)

`sources` (generalizes `channels`)

A configured place we ingest from. Maps 1:1 to today's per-account `channels` rows for Telegram; future providers add their own source kinds.

```
sources (
    id                INTEGER PRIMARY KEY AUTOINCREMENT,
    account_id        INTEGER NOT NULL REFERENCES accounts(id),
    provider          TEXT NOT NULL DEFAULT 'telegram',
    provider_source_id TEXT NOT NULL,        -- Telegram channel_id; future: WA group id
    title             TEXT,
    kind              TEXT NOT NULL DEFAULT 'channel',  -- 'channel' | 'group' | 'feed'
    watched           INTEGER NOT NULL DEFAULT 0,       -- ADR 0009 selective scanning
    seen_first_at     TEXT NOT NULL,
    last_seen_at      TEXT,
    UNIQUE (account_id, provider, provider_source_id)
);
```

`channels.user_id` (now ? `accounts.id`) maps to `sources.account_id`; `channels.channel_id` maps to `sources.provider_source_id`. The `watched` flag (ADR 0009, deferred to Track E) is included here so Track E doesn't need another migration.

`content_items` (generalizes `videos`)

A normalized item discovered from any source. Phase 1 stores only videos (matches today's `videos`); the columns are provider-neutral but the Phase-1 migration only populates video fields.

```
content_items (
    id                INTEGER PRIMARY KEY AUTOINCREMENT,
    account_id        INTEGER NOT NULL REFERENCES accounts(id),
    source_id         INTEGER NOT NULL REFERENCES sources(id),
    provider          TEXT NOT NULL DEFAULT 'telegram',
    provider_message_id TEXT NOT NULL,       -- Telegram message_id; future: WA message
id
    content_type      TEXT NOT NULL DEFAULT 'video',  -- 'video' | 'link' | 'article' |
...
    text              TEXT,                  -- caption / post text (nullable)
    link_url          TEXT,                  -- for link/article posts
    media_ref         TEXT,                  -- provider media reference (file path,
blob id)
    document_id       TEXT,                  -- Telegram document.id (kept for TG-
specific lookups)
    duration_s        INTEGER,
    width             INTEGER,
    height            INTEGER,
    size_bytes        INTEGER,
    mime_type         TEXT,
    dedup_key         TEXT NOT NULL,         -- ADR 0011 Question 1 (Option 1A)
    matched           INTEGER NOT NULL DEFAULT 0,
```

```
        seen_at           TEXT NOT NULL,
        -- Tier-0 equivalent: provider_message_id is only unique within a source
        -- (a Telegram message_id is per-channel), so source_id is required to
        -- avoid false collisions across sources for the same account.
        UNIQUE (account_id, source_id, provider_message_id),
        UNIQUE (account_id, dedup_key)                        -- Tier-1/2 equivalent (Option
1A)
    );
    ```

    Note on `forwarded`: dropped from `content_items`. Per-destination
    delivery state moves to `delivery_attempts` (Track C, #28). For Phase 1
    (before Track C lands), `content_items.matched` alone is sufficient ?
    delivery still goes to the single `delivery_target`, and the boolean
    `forwarded` is retained temporarily on a *deprecated* `videos`-compat view
    so the v1 delivery path keeps working. Track C removes it.

    ### `policies` (generalizes `filter_config`) ? Option 2A

    As specified in Question 2 above.

    ### `destinations` (new ? Track C will populate)

    Defined here so the schema is complete, but **Track C (#28) implements the
    delivery layer**. Phase-1 presence is the table only (no delivery logic).

    ```
    destinations (
        id          INTEGER PRIMARY KEY AUTOINCREMENT,
        account_id  INTEGER NOT NULL REFERENCES accounts(id),
        provider    TEXT NOT NULL DEFAULT 'telegram',
        target      TEXT NOT NULL,           -- 'me', '@username', chat id (ADR 0005)
        kind        TEXT NOT NULL DEFAULT 'saved',  -- 'saved' | 'channel' | 'chat'
        title       TEXT,
        is_output   INTEGER NOT NULL DEFAULT 0,      -- ADR 0009: distinguishes output
channels
        created_at  TEXT NOT NULL,
        UNIQUE (account_id, provider, target)
    );
    ```

    ### `routing_rules` (new ? Track C will populate)

    Per ADR 0009 (canonical name locked in issue #25).

    ```
    routing_rules (
        id             INTEGER PRIMARY KEY AUTOINCREMENT,
        account_id     INTEGER NOT NULL REFERENCES accounts(id),
        source_id      INTEGER NOT NULL REFERENCES sources(id),
        policy_id      INTEGER,               -- nullable ? use account's default policy
        destination_id INTEGER NOT NULL REFERENCES destinations(id),
        enabled        INTEGER NOT NULL DEFAULT 1,
        priority       INTEGER NOT NULL DEFAULT 0,
        created_at     TEXT NOT NULL,
        UNIQUE (account_id, source_id, destination_id)
    );
    ```

    ### `delivery_attempts` (new ? Track C will populate)

    Per #28's idempotency requirement (unique index on
    `(content_item_id, destination_id)`).

    ```
```

```
    delivery_attempts (
        id              INTEGER PRIMARY KEY AUTOINCREMENT,
        content_item_id INTEGER NOT NULL REFERENCES content_items(id),
        destination_id  INTEGER NOT NULL REFERENCES destinations(id),
        route_id        INTEGER REFERENCES routing_rules(id),
        status          TEXT NOT NULL DEFAULT 'pending',  --
'pending'|'delivered'|'failed'|'retrying'
        attempt_count   INTEGER NOT NULL DEFAULT 0,
        last_error      TEXT,
        created_at      TEXT NOT NULL,
        delivered_at    TEXT,
        next_retry_at   TEXT,
        UNIQUE (content_item_id, destination_id)         -- ADR #28 idempotency
    );
    ```

    ## Migration strategy (integer-preserving, phased)

    This track lands **`sources`, `content_items`, and `policies`** only.
    `destinations`, `routing_rules`, `delivery_attempts` are defined in the
    schema (so the ADR is complete) but their tables are created empty and
    populated by Track C. This keeps Track B focused and reviewable.

    ### Migration `_migration_003_domain_schema` (idempotent)

    1. **Create** `sources`, `content_items`, `policies`, `destinations`,
       `routing_rules`, `delivery_attempts` (all `IF NOT EXISTS`).
    2. **Backfill `sources`** from `channels`:
       `provider='telegram'`, `provider_source_id=channel_id`, `watched=1`
       (ADR 0009 backfill: already-seen channels stay watched).
    3. **Backfill `content_items`** from `videos`, computing `dedup_key` via the
       explicit decision tree from Option 1A (every branch reachable, no
       `COALESCE(...,0)` collapsing partial metadata to a degenerate key):
       - If `document_id IS NOT NULL`:
         `dedup_key = 'tg:doc:' || document_id`.
       - Else if `size_bytes IS NOT NULL AND duration_s IS NOT NULL AND height IS NOT NULL
AND width IS NOT NULL`:
         `dedup_key = 'tg:fp:' || size_bytes || ':' || duration_s || ':' || height || ':' ||
width`.
       - Else (Tier-0 fallback; partial or missing media metadata):
         `dedup_key = 'tg:msg:' || source_id || ':' || provider_message_id`,
         where `source_id` is the joined `sources.id` for the row's legacy
         `channel_id` and `provider_message_id` is the legacy `message_id`.

       The implementation PR must include migration tests that seed rows hitting
       each branch (including all-fingerprint-fields-null) and assert no two
       rows collapse to the same `dedup_key`.
    4. **Backfill `policies`** from `filter_config`: one row per account,
       `name='default'`, `is_default=1`, `config_json` copied verbatim.
    5. **Rename** (don't drop) `channels` ? `channels_deprecated`,
       `videos` ? `videos_deprecated`, `filter_config` ? `filter_config_deprecated`.
       Kept for one release as a rollback safety net (ADR 0010 pattern).

    The legacy `videos`/`channels`/`filter_config` tables and their
    `db.load_filter_config` / `upsert_channel` / `insert_video` repos stay
    working as shims over the new tables (same pattern as the Track-A `User`
    shim), so existing call sites in `telethon_client.py`, `backfill.py`, and
    `commands.py` keep working without a flag day.

    ## Consequences

    Positive
    - Provider-neutral schema: `sources`, `content_items`, `policies`,
      `destinations`, `routing_rules`, `delivery_attempts` all carry a
      `provider` column or provider-prefixed keys. Adding WhatsApp later is a
```
```

```
364       new `provider='whatsapp'` value, not a new schema.
365     - `dedup_key` (Option 1A) generalizes ADR 0006 without losing any dedup
366       quality and without downloading anything.
367     - `Policy` (Option 2A) gives a single home for filter + future enrichment
368       config, enabling ADR 0009's per-route overrides with no second migration.
369     - Backwards-compatible via shims, mirroring the Track-A pattern that
370       already passed review.
371
372     **Negative**
373     - Six new tables; the schema grows significantly. Each is small and
374       single-purpose, but the surface area is larger.
375     - The dedup-key derivation logic is in code, not the schema ? needs its
376       own unit tests (a `compute_dedup_key(provider, **metadata)` pure function).
377     - Phase 1 carries some dead-on-arrival columns (`content_items.link_url`,
378       `text`) that aren't populated until later tracks. This is intentional
379       (avoid future migrations) but means some columns are NULL until then.
380
381     ## Alternatives considered
382
383     - **Do nothing; keep `channels`/`videos`/`filter_config` and special-case
384       providers in code.** Rejected ? same conflation argument as ADR 0010.
385       Telegram-specific ids in column names (`channel_id`, `message_id`,
386       `document_id`) would leak into every future provider's code path.
387     - **Land all six tables' *logic* in this track.** Rejected ? Track B is
388       already the largest track. Splitting schema (Track B) from delivery
389       logic (Track C) keeps each PR reviewable. The empty `destinations` /
390       `routing_rules` / `delivery_attempts` tables in this track are the
391       contract Track C codes against.
392     - **`content_items` as a view over `videos` (no migration).** Rejected ?
393       views don't give us the `provider` column or the `dedup_key` index, and
394       SQLite view support for writes is limited. A real table is needed.
395
396     ## Open questions for the decision owner
397
398     1. **Question 1** ? `dedup_key`: 1A (tiered, recommended), 1B (hash,
399        rejected), or 1C (provider-assigned only).
400     2. **Question 2** ? `Policy` ? `filter_config`: 2A (generalize, recommended)
401        or 2B (wrap, rejected).
402     3. **Dead columns in Phase 1** ? acceptable to land `content_items.link_url`
403        / `text` as NULL until later tracks populate them? Recommendation: yes
404        (avoids a future `ALTER TABLE`).
405     4. **Deprecated-table retention** ? one release (default), matching ADR 0010?
406        Recommendation: yes.
407
408     ## References
409
410     - ADR 0003 ? metadata-first probing (the no-download constraint that rules
411       out Option 1B's content hashing).
412     - ADR 0005 ? per-user delivery target (generalized by `destinations` /
413       `routing_rules`).
414     - ADR 0006 ? content dedup (generalized by `content_items.dedup_key`,
415       Option 1A).
416     - ADR 0009 ? selective scanning + multiplexed routing (depends on
417       `sources.watched` and `routing_rules`, both defined here).
418     - ADR 0010 ? identity/session separation (this ADR's `account_id` FKs point
419       at `accounts.id`).
420     - Issue #27 ? Track B (this ADR's tracking issue).
421     - Issue #25 ? umbrella; defines the six concepts and the implementation order.
422
```

4. user (2026-07-06T06:02:59.724Z)

```
1       """Tests for the SQLite data layer (db.py) using an in-memory database."""
2
3       from __future__ import annotations
```

```

    import sqlite3

    import aiosqlite
    import pytest

    from tg_video_filter import db
    from tg_video_filter.models import (
        DEFAULT_FILTER_CONFIG,
        Account,
        Channel,
        ContentItem,
        FilterConfig,
        Policy,
        Source,
        TelegramSession,
        User,
        VideoRecord,
    )

    USER_ID = 111
    CHANNEL_ID = -1001234567890


    @pytest.fixture
    async def conn() -> aiosqlite.Connection:
        """An in-memory SQLite connection with the schema applied."""
        c = await aiosqlite.connect(":memory:")
        await db._apply_schema(c)
        yield c
        await c.close()


    async def _seed_user(conn: aiosqlite.Connection, uid: int = USER_ID) -> User:
        return await db.upsert_user(conn, User(id=uid, phone="+15555550100"))


    # --- schema / init -------------------------------------------------------


    async def test_init_db_creates_tables(tmp_path: object) -> None:
        db_path = str(tmp_path) + "/test.db"  # type: ignore[operator]
        await db.init_db(db_path)
        # Re-open and confirm all expected tables exist. Post-ADR-0011 the
        # provider-neutral tables are sources/content_items/policies (plus the
        # empty Track-C tables destinations/routing_rules/delivery_attempts);
        # the legacy channels/videos/filter_config are absent on fresh DBs
        # (they only exist as *_deprecated on migrated legacy DBs).
        async with (
            aiosqlite.connect(db_path) as c,
            c.execute("SELECT name FROM sqlite_master WHERE type='table' ORDER BY name") as
cur,
        ):
            names = [r[0] for r in await cur.fetchall()]
        assert set(names) >= {
            "accounts",
            "telegram_sessions",
            "sources",
            "content_items",
            "policies",
            "destinations",
            "routing_rules",
            "delivery_attempts",
        }

```

```

    # --- user repos ------------------------------------------------------------


    async def test_upsert_and_get_user(conn: aiosqlite.Connection) -> None:
        created = await _seed_user(conn)
        assert created.id == USER_ID
        assert created.enabled is True

        fetched = await db.get_user(conn, USER_ID)
        assert fetched is not None
        assert fetched.phone == "+15555550100"


    async def test_get_user_returns_none_when_absent(conn: aiosqlite.Connection) -> None:
        assert await db.get_user(conn, 999999) is None


    async def test_upsert_user_updates_existing(conn: aiosqlite.Connection) -> None:
        await _seed_user(conn)
        # Disable the user via an upsert.
        await db.upsert_user(conn, User(id=USER_ID, phone="+15555550200", enabled=False))
        fetched = await db.get_user(conn, USER_ID)
        assert fetched is not None
        assert fetched.enabled is False
        assert fetched.phone == "+15555550200"


    async def test_list_enabled_users_filters_disabled(conn: aiosqlite.Connection) -> None:
        await db.upsert_user(conn, User(id=1, enabled=True))
        await db.upsert_user(conn, User(id=2, enabled=False))
        await db.upsert_user(conn, User(id=3, enabled=True))
        enabled = await db.list_enabled_users(conn)
        assert sorted(u.id for u in enabled) == [1, 3]


    # --- account + session repos (ADR 0010) ------------------------------------


    async def _seed_account(conn: aiosqlite.Connection, aid: int = USER_ID) -> Account:
        return await db.upsert_account(conn, Account(id=aid, telegram_id=aid))


    async def test_upsert_and_get_account(conn: aiosqlite.Connection) -> None:
        created = await db.upsert_account(
            conn, Account(id=USER_ID, telegram_id=USER_ID, display_name="Alice")
        )
        assert created.id == USER_ID
        assert created.telegram_id == USER_ID
        assert created.provider == "telegram"

        fetched = await db.get_account(conn, USER_ID)
        assert fetched is not None
        assert fetched.display_name == "Alice"


    async def test_get_account_returns_none_when_absent(conn: aiosqlite.Connection) -> None:
        assert await db.get_account(conn, 999999) is None


    async def test_get_account_by_telegram_id_resolves_bot_from_id(
        conn: aiosqlite.Connection,
    ) -> None:
        """The Bot API authorization bridge: from.id ? Account (ADR 0008/0010)."""
        await db.upsert_account(conn, Account(id=USER_ID, telegram_id=USER_ID))
```

```
133         found = await db.get_account_by_telegram_id(conn, USER_ID)
134         assert found is not None
135         assert found.id == USER_ID
136         # Unknown telegram id resolves to None.
137         assert await db.get_account_by_telegram_id(conn, 888888) is None
138 
139 
140     async def test_upsert_and_get_telegram_session(conn: aiosqlite.Connection) -> None:
141         await _seed_account(conn)
142         session = await db.upsert_telegram_session(
143             conn,
144             TelegramSession(
145                 account_id=USER_ID,
146                 phone="+15555550100",
147                 session_path="user_111.session",
148                 enabled=True,
149             ),
150         )
151         assert session.account_id == USER_ID
152         assert session.session_path == "user_111.session"
153         assert session.enabled is True
154 
155         fetched = await db.get_telegram_session(conn, USER_ID)
156         assert fetched is not None
157         assert fetched.phone == "+15555550100"
158 
159 
160     async def test_list_runnable_sessions_excludes_disabled_and_bot_only(
161         conn: aiosqlite.Connection,
162     ) -> None:
163         """ADR 0010 structural invariant: bot-only + disabled sessions are invisible."""
164         # Account 1: userbot, enabled ? runnable.
165         await db.upsert_account(conn, Account(id=1, telegram_id=1))
166         await db.upsert_telegram_session(
167             conn, TelegramSession(account_id=1, session_path="user_1.session", enabled=True)
168         )
169         # Account 2: userbot, disabled ? not runnable.
170         await db.upsert_account(conn, Account(id=2, telegram_id=2))
171         await db.upsert_telegram_session(
172             conn, TelegramSession(account_id=2, session_path="user_2.session",
enabled=False)
173         )
174         # Account 3: bot-only (no session row) ? not runnable.
175         await db.upsert_account(conn, Account(id=3, telegram_id=3))
176 
177         pairs = await db.list_runnable_sessions(conn)
178         assert [acct.id for acct, _ in pairs] == [1]
179 
180 
181     async def test_bot_only_account_has_no_session_and_is_not_a_user(
182         conn: aiosqlite.Connection,
183     ) -> None:
184         """A bot-only account is a real Account but not a legacy 'enabled user'."""
185         await db.upsert_account(conn, Account(id=USER_ID, telegram_id=USER_ID))
186 
187         # Account exists.
188         assert await db.get_account(conn, USER_ID) is not None
189         # No session row.
190         assert await db.get_telegram_session(conn, USER_ID) is None
191         # Legacy User shim returns a user with enabled=False (not runnable).
192         legacy = await db.get_user(conn, USER_ID)
193         assert legacy is not None
194         assert legacy.enabled is False
195         # And it is excluded from the enabled-users list.
196         assert await db.list_enabled_users(conn) == []
```

```
197
198
199     # --- channel repos ------------------------------------------------------------
200
201
202     async def test_upsert_channel_inserts_and_is_idempotent(conn: aiosqlite.Connection) ->
None:
203         await _seed_user(conn)
204         ch = await db.upsert_channel(
205             conn, Channel(user_id=USER_ID, channel_id=CHANNEL_ID, title="First title")
206         )
207         assert ch.id is not None
208         assert ch.title == "First title"
209
210         # Second upsert with a new title updates the existing row (no duplicate).
211         ch2 = await db.upsert_channel(
212             conn, Channel(user_id=USER_ID, channel_id=CHANNEL_ID, title="New title")
213         )
214         assert ch2.id == ch.id
215         assert ch2.title == "New title"
216
217
218     # --- video repos --------------------------------------------------------------
219
220
221     async def test_get_video_absent(conn: aiosqlite.Connection) -> None:
222         assert await db.get_video(conn, USER_ID, CHANNEL_ID, 1) is None
223
224
225     async def test_insert_and_get_video_round_trip(conn: aiosqlite.Connection) -> None:
226         await _seed_user(conn)
227         v = VideoRecord(
228             user_id=USER_ID,
229             channel_id=CHANNEL_ID,
230             message_id=42,
231             duration_s=120,
232             width=1920,
233             height=1080,
234             size_bytes=5_000_000,
235             mime_type="video/mp4",
236             matched=True,
237             forwarded=False,
238         )
239         inserted = await db.insert_video(conn, v)
240         assert inserted.id is not None
241
242         fetched = await db.get_video(conn, USER_ID, CHANNEL_ID, 42)
243         assert fetched is not None
244         assert fetched.duration_s == 120
245         assert fetched.height == 1080
246         assert fetched.size_bytes == 5_000_000
247         assert fetched.mime_type == "video/mp4"
248         assert fetched.matched is True
249         assert fetched.forwarded is False
250
251
252     async def test_insert_video_dedup_collision(conn: aiosqlite.Connection) -> None:
253         await _seed_user(conn)
254         v = VideoRecord(user_id=USER_ID, channel_id=CHANNEL_ID, message_id=7)
255         await db.insert_video(conn, v)
256         # Same (user, channel, message) must violate the unique constraint.
257         with pytest.raises(sqlite3.IntegrityError):
258             await db.insert_video(conn, v)
259
260
```

```
    async def test_mark_forwarded_is_noop_post_adr_0011(conn: aiosqlite.Connection) -> None:
        """ADR 0011: mark_forwarded is a no-op shim (delivery state ? Track C).

        Post-migration-003 there is no ``forwarded`` column on ``content_items``;
        per-destination delivery state moves to ``delivery_attempts`` in Track C.
        For Phase 1 the call must succeed without error (existing callers in
        ``delivery.py`` keep working) and the surviving ``matched`` flag is
        unaffected.
        """
        await _seed_user(conn)
        inserted = await db.insert_video(
            conn, VideoRecord(user_id=USER_ID, channel_id=CHANNEL_ID, message_id=9,
matched=True)
        )
        assert inserted.id is not None
        # No-op: must not raise.
        await db.mark_forwarded(conn, inserted.id)
        fetched = await db.get_video(conn, USER_ID, CHANNEL_ID, 9)
        assert fetched is not None
        # matched survives; forwarded is always False post-ADR-0011.
        assert fetched.matched is True
        assert fetched.forwarded is False


    async def test_list_videos_for_user_orders_desc(conn: aiosqlite.Connection) -> None:
        await _seed_user(conn)
        for msg_id in (1, 2, 3):
            await db.insert_video(
                conn, VideoRecord(user_id=USER_ID, channel_id=CHANNEL_ID, message_id=msg_id)
            )
        vids = await db.list_videos_for_user(conn, USER_ID)
        # Newest insert first (DESC by id).
        assert [v.message_id for v in vids] == [3, 2, 1]


    async def test_video_record_dedup_key() -> None:
        v = VideoRecord(user_id=1, channel_id=2, message_id=3)
        assert v.dedup_key == (1, 2, 3)


    async def test_video_record_content_fingerprint() -> None:
        v = VideoRecord(
            user_id=1,
            channel_id=2,
            message_id=3,
            size_bytes=5_000_000,
            duration_s=120,
            height=1080,
            width=1920,
        )
        assert v.content_fingerprint == (5_000_000, 120, 1080, 1920)


    async def test_video_record_content_fingerprint_none_when_missing_field() -> None:
        """Fingerprint is None if any of the four metadata fields is missing."""
        v = VideoRecord(
            user_id=1,
            channel_id=2,
            message_id=3,
            size_bytes=5_000_000,
            duration_s=120,
            height=1080,
            # width is None
        )
        assert v.content_fingerprint is None
```

```
325
326
327     # --- content dedup (Tiers 1 & 2) ----------------------------------------
328
329
330     async def test_get_video_by_document_id_hit(conn: aiosqlite.Connection) -> None:
331         await _seed_user(conn)
332         await db.insert_video(
333             conn,
334             VideoRecord(
335                 user_id=USER_ID,
336                 channel_id=CHANNEL_ID,
337                 message_id=1,
338                 document_id=555000111,
339             ),
340         )
341         found = await db.get_video_by_document_id(conn, USER_ID, 555000111)
342         assert found is not None
343         assert found.document_id == 555000111
344         assert found.message_id == 1
345
346
347     async def test_get_video_by_document_id_miss(conn: aiosqlite.Connection) -> None:
348         await _seed_user(conn)
349         assert await db.get_video_by_document_id(conn, USER_ID, 999999) is None
350
351
352     async def test_insert_video_document_id_collision(conn: aiosqlite.Connection) -> None:
353         """Same document_id for the same user ? IntegrityError (Tier-1 dedup at DB
level)."""
354         await _seed_user(conn)
355         await db.insert_video(
356             conn,
357             VideoRecord(
358                 user_id=USER_ID,
359                 channel_id=CHANNEL_ID,
360                 message_id=10,
361                 document_id=123456789,
362             ),
363         )
364         # Same doc_id, different channel/message ? must collide on the index.
365         with pytest.raises(sqlite3.IntegrityError):
366             await db.insert_video(
367                 conn,
368                 VideoRecord(
369                     user_id=USER_ID,
370                     channel_id=-100999999,
371                     message_id=99,
372                     document_id=123456789,
373                 ),
374             )
375
376
377     async def test_insert_video_null_document_id_no_collision(conn: aiosqlite.Connection) ->
None:
378         """Multiple rows with document_id=NULL do NOT collide (partial unique index)."""
379         await _seed_user(conn)
380         await db.insert_video(
381             conn,
382             VideoRecord(
383                 user_id=USER_ID,
384                 channel_id=CHANNEL_ID,
385                 message_id=20,
386                 document_id=None,
387             ),
```

```
        )
        # Same user, no document_id, different message ? should succeed.
        inserted = await db.insert_video(
            conn,
            VideoRecord(
                user_id=USER_ID,
                channel_id=CHANNEL_ID,
                message_id=21,
                document_id=None,
            ),
        )
        assert inserted.id is not None


    async def test_get_video_by_content_fingerprint_hit(conn: aiosqlite.Connection) -> None:
        await _seed_user(conn)
        await db.insert_video(
            conn,
            VideoRecord(
                user_id=USER_ID,
                channel_id=CHANNEL_ID,
                message_id=5,
                size_bytes=10_000_000,
                duration_s=180,
                height=720,
                width=1280,
            ),
        )
        found = await db.get_video_by_content_fingerprint(
            conn,
            USER_ID,
            10_000_000,
            180,
            720,
            1280,
        )
        assert found is not None
        assert found.message_id == 5


    async def test_get_video_by_content_fingerprint_miss(conn: aiosqlite.Connection) ->
None:
        await _seed_user(conn)
        # No video inserted ? should be None.
        assert (
            await db.get_video_by_content_fingerprint(
                conn,
                USER_ID,
                1,
                1,
                1,
                1,
            )
            is None
        )


    # --- filter config repos ---------------------------------------------------


    async def test_load_filter_config_defaults_when_absent(conn: aiosqlite.Connection) ->
None:
        await _seed_user(conn)
        cfg = await db.load_filter_config(conn, USER_ID)
        assert cfg == DEFAULT_FILTER_CONFIG
```

```
451
452
453     async def test_save_and_load_filter_config_round_trip(conn: aiosqlite.Connection) ->
None:
454         await _seed_user(conn)
455         custom = FilterConfig(
456             min_duration_s=10, max_duration_s=3600, target_height=720, max_size_mb=1024
457         )
458         await db.save_filter_config(conn, USER_ID, custom)
459         loaded = await db.load_filter_config(conn, USER_ID)
460         assert loaded == custom
461
462
463     async def test_save_filter_config_overwrites(conn: aiosqlite.Connection) -> None:
464         await _seed_user(conn)
465         await db.save_filter_config(conn, USER_ID, FilterConfig(max_duration_s=100))
466         await db.save_filter_config(conn, USER_ID, FilterConfig(max_duration_s=200))
467         loaded = await db.load_filter_config(conn, USER_ID)
468         assert loaded.max_duration_s == 200
469
470
471     async def test_filter_config_with_null_bounds_round_trips(conn: aiosqlite.Connection) ->
None:
472         """None bounds (rule disabled) must survive the JSON round trip."""
473         await _seed_user(conn)
474         cfg = FilterConfig(
475             min_duration_s=None, max_duration_s=None, target_height=None, max_size_mb=None
476         )
477         await db.save_filter_config(conn, USER_ID, cfg)
478         loaded = await db.load_filter_config(conn, USER_ID)
479         assert loaded.min_duration_s is None
480         assert loaded.max_duration_s is None
481         assert loaded.target_height is None
482         assert loaded.max_size_mb is None
483
484
485     async def test_delivery_mode_round_trips(conn: aiosqlite.Connection) -> None:
486         """The delivery_mode field survives the JSON round trip for all values."""
487         from tg_video_filter.models import DeliveryMode
488
489         await _seed_user(conn)
490         for mode in DeliveryMode:
491             await db.save_filter_config(conn, USER_ID, FilterConfig(delivery_mode=mode))
492             loaded = await db.load_filter_config(conn, USER_ID)
493             assert loaded.delivery_mode == mode
494
495
496     async def test_filter_config_rejects_inverted_duration_bounds() -> None:
497         """A config with min_duration > max_duration must raise ValidationError."""
498         from pydantic import ValidationError
499
500         from tg_video_filter.models import FilterConfig
501
502         with pytest.raises(ValidationError, match="min_duration_s.*must be <="):
503             FilterConfig(min_duration_s=7200, max_duration_s=5)
504
505
506     def test_filter_config_accepts_equal_min_max_duration() -> None:
507         """min_duration_s == max_duration_s is valid (exact-match filter)."""
508         from tg_video_filter.models import FilterConfig
509
510         cfg = FilterConfig(min_duration_s=60, max_duration_s=60)
511         assert cfg.min_duration_s == 60
512         assert cfg.max_duration_s == 60
513
```

```

    async def test_old_config_without_delivery_mode_defaults_to_forward(
        conn: aiosqlite.Connection,
    ) -> None:
        """A pre-v1.1 config JSON (no delivery_mode key) must default to FORWARD."""
        await _seed_user(conn)
        # Simulate an old row by inserting raw JSON without the delivery_mode field
        # into the default policy (ADR 0011: filter_config is now policies).
        old_json =
'{"min_duration_s":5,"max_duration_s":7200,"target_height":1080,"max_size_mb":2048}'
        await conn.execute(
            "INSERT OR REPLACE INTO policies (account_id, name, config_json, is_default, "
            "created_at, updated_at) VALUES (?, 'default', ?, 1, CURRENT_TIMESTAMP,
CURRENT_TIMESTAMP)",
            (USER_ID, old_json),
        )
        await conn.commit()
        loaded = await db.load_filter_config(conn, USER_ID)
        from tg_video_filter.models import DeliveryMode

        assert loaded.delivery_mode == DeliveryMode.FORWARD


    async def test_delivery_target_round_trips(conn: aiosqlite.Connection) -> None:
        """The delivery_target field survives the JSON round trip for all forms."""
        await _seed_user(conn)
        for target in ("me", "@my-feed", "-1001234567890", "1234567890"):
            await db.save_filter_config(conn, USER_ID, FilterConfig(delivery_target=target))
            loaded = await db.load_filter_config(conn, USER_ID)
            assert loaded.delivery_target == target


    async def test_old_config_without_delivery_target_defaults_to_me(
        conn: aiosqlite.Connection,
    ) -> None:
        """A pre-v1.2 config JSON (no delivery_target key) must default to 'me'."""
        await _seed_user(conn)
        old_json = (
            '{"min_duration_s":5,"max_duration_s":7200,"target_height":1080,'
            '"max_size_mb":2048,"delivery_mode":"forward"}'
        )
        await conn.execute(
            "INSERT OR REPLACE INTO policies (account_id, name, config_json, is_default, "
            "created_at, updated_at) VALUES (?, 'default', ?, 1, CURRENT_TIMESTAMP,
CURRENT_TIMESTAMP)",
            (USER_ID, old_json),
        )
        await conn.commit()
        loaded = await db.load_filter_config(conn, USER_ID)
        assert loaded.delivery_target == "me"


    def test_filter_config_coerces_int_delivery_target() -> None:
        """An int chat id is coerced to str (so JSON stays uniform)."""
        from tg_video_filter.models import FilterConfig

        cfg = FilterConfig(delivery_target=-1001234567890)  # type: ignore[arg-type]
        assert cfg.delivery_target == "-1001234567890"


    def test_filter_config_rejects_empty_delivery_target() -> None:
        """An empty delivery_target is invalid (min_length=1)."""
        from pydantic import ValidationError

        from tg_video_filter.models import FilterConfig
```

```
576
577         with pytest.raises(ValidationError):
578             FilterConfig(delivery_target="")
579
580
581     async def test_pragmas_are_set_on_init(conn: aiosqlite.Connection) -> None:
582         """Verify WAL mode, foreign keys, and busy timeout are enabled."""
583         await db._apply_schema(conn)
584         async with conn.execute("PRAGMA journal_mode") as cur:
585             row = await cur.fetchone()
586             assert row is not None
587             # In-memory databases report "memory"; file-backed DBs report "wal".
588             assert row[0].lower() in ("wal", "memory")
589         async with conn.execute("PRAGMA foreign_keys") as cur:
590             assert (await cur.fetchone())[0] == 1
591         async with conn.execute("PRAGMA busy_timeout") as cur:
592             assert (await cur.fetchone())[0] == 5000
593
594
595     async def test_foreign_key_enforcement_rejects_orphan_video(
596         conn: aiosqlite.Connection,
597     ) -> None:
598         """Inserting a video for a non-existent user must fail with FK error."""
599         import sqlite3
600
601         with pytest.raises(sqlite3.IntegrityError, match="FOREIGN KEY"):
602             await db.insert_video(conn, VideoRecord(user_id=99999, channel_id=1,
message_id=1))
603
604
605     # --- migrations ----------------------------------------------------------
606
607
608     async def test_fresh_db_has_document_id_column(conn: aiosqlite.Connection) -> None:
609         """A fresh schema (via _apply_schema) must include document_id on content_items.
610
611         ADR 0011: the partial unique index ``idx_videos_user_document`` is
612         superseded by ``UNIQUE(account_id, dedup_key)`` on ``content_items``; the
613         ``document_id`` column survives as a TEXT field for TG-specific Tier-1
614         lookups. This test now checks the new location.
615         """
616         assert await db._column_exists(conn, "content_items", "document_id")
617         # The provider-neutral dedup uniqueness lives on content_items, not videos.
618         async with conn.execute("PRAGMA index_list('content_items')") as cur:
619             idx_names = {row[1] for row in await cur.fetchall()}
620         # SQLite names the UNIQUE(...) constraints auto-generated indexes; assert
621         # at least one unique index exists (the dedup_key uniqueness).
622         assert any("dedup_key" in idx or "sqlite_autoindex" in idx for idx in idx_names),
idx_names
623
624
625     async def test_fresh_db_user_version_is_latest(conn: aiosqlite.Connection) -> None:
626         """init_db/_apply_schema must bump user_version to the highest migration."""
627         await db._apply_schema(conn)
628         async with conn.execute("PRAGMA user_version") as cur:
629             version = (await cur.fetchone())[0]
630         assert version == len(db._MIGRATIONS)
631         assert version >= 1
632
633
634     async def test_migration_adds_document_id_to_legacy_db(tmp_path: object) -> None:
635         """A v1 DB (no document_id, user_version=0) must be migrated on open."""
636         db_path = str(tmp_path) + "/legacy.db"  # type: ignore[operator]
637
638         # Build a legacy v1 schema WITH the real FK definitions (REFERENCES users),
```

```
        # no document_id column, user_version = 0. Using the real FK constraints
        # is what lets this test catch the PR #33 regression where SQLite rewrites
        # FK references to follow a rename (users ? users_deprecated) instead of
        # retargeting them to accounts.
        legacy_schema = """
        CREATE TABLE users (
            id INTEGER PRIMARY KEY, phone TEXT,
            enabled INTEGER NOT NULL DEFAULT 1, created_at TEXT NOT NULL
        );
        CREATE TABLE channels (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            user_id INTEGER NOT NULL REFERENCES users(id),
            channel_id INTEGER NOT NULL, title TEXT, seen_first_at TEXT NOT NULL,
            UNIQUE (user_id, channel_id)
        );
        CREATE TABLE videos (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            user_id INTEGER NOT NULL REFERENCES users(id),
            channel_id INTEGER NOT NULL, message_id INTEGER NOT NULL,
            duration_s INTEGER, width INTEGER, height INTEGER, size_bytes INTEGER,
            mime_type TEXT, matched INTEGER NOT NULL DEFAULT 0,
            forwarded INTEGER NOT NULL DEFAULT 0, seen_at TEXT NOT NULL,
            UNIQUE (user_id, channel_id, message_id)
        );
        CREATE TABLE filter_config (
            user_id INTEGER PRIMARY KEY REFERENCES users(id), config_json TEXT NOT NULL
        );
        """
        async with aiosqlite.connect(db_path) as conn:
            await conn.executescript(legacy_schema)
            # Seed rows so we can confirm data survives the migration.
            await conn.execute(
                "INSERT INTO users (id, phone, enabled, created_at) VALUES (111, '+1', 1,
'2024-01-01T00:00:00Z')"
            )
            await conn.execute(
                "INSERT INTO channels (user_id, channel_id, title, seen_first_at) "
                "VALUES (111, -1001, 'pre-migration', '2024-01-01T00:00:00Z')"
            )
            await conn.execute(
                "INSERT INTO videos (user_id, channel_id, message_id, duration_s, width,
height, "
                "size_bytes, mime_type, matched, forwarded, seen_at) "
                "VALUES (111, -1001, 1, 60, 1920, 1080, 1000000, 'video/mp4', 1, 1,
'2024-01-01T00:00:00Z')"
            )
            await conn.execute("INSERT INTO filter_config (user_id, config_json) VALUES
(111, '{}')")
            await conn.commit()
            # user_version is 0 on a freshly created DB.

        # Now open via init_db ? migrations 001, 002, and 003 must run.
        await db.init_db(db_path)

        async with aiosqlite.connect(db_path) as conn:
            await conn.execute("PRAGMA foreign_keys=ON")
            # Column added (migration 001) ? on the legacy videos table, now renamed.
            assert await db._column_exists(conn, "videos_deprecated", "document_id")
            # Identity split (migration 002): users renamed, accounts present.
            assert await db._table_exists(conn, "accounts")
            assert await db._table_exists(conn, "telegram_sessions")
            assert await db._table_exists(conn, "users_deprecated")
            assert not await db._table_exists(conn, "users")
            # accounts.id copied from users.id (integer-preserving).
            async with conn.execute("SELECT id, telegram_id, provider FROM accounts") as
```

```
cur:
                rows = await cur.fetchall()
            assert rows == [(111, 111, "telegram")]
            # telegram_sessions row derived from the legacy user row.
            async with conn.execute(
                "SELECT account_id, phone, session_path, enabled FROM telegram_sessions"
            ) as cur:
                srows = await cur.fetchall()
            assert srows == [(111, "+1", "user_111.session", 1)]

            # Domain schema (migration 003): legacy tables renamed to *_deprecated,
            # provider-neutral tables backfilled from them.
            assert await db._table_exists(conn, "sources")
            assert await db._table_exists(conn, "content_items")
            assert await db._table_exists(conn, "policies")
            assert await db._table_exists(conn, "channels_deprecated")
            assert await db._table_exists(conn, "videos_deprecated")
            assert await db._table_exists(conn, "filter_config_deprecated")
            assert not await db._table_exists(conn, "channels")
            assert not await db._table_exists(conn, "videos")
            assert not await db._table_exists(conn, "filter_config")

            # FK contract (PR #33 review): the rebuilt legacy tables (now
            # *_deprecated) reference accounts(id), NOT users_deprecated(id).
            for tbl in ("channels_deprecated", "videos_deprecated",
"filter_config_deprecated"):
                async with conn.execute(f"PRAGMA foreign_key_list({tbl})") as cur:
                    fk_rows = await cur.fetchall()
                referenced_tables = {r[2] for r in fk_rows}
                assert "accounts" in referenced_tables, f"{tbl} FK does not reference
accounts"
                assert (
                    "users_deprecated" not in referenced_tables
                ), f"{tbl} FK still references users_deprecated (rename-rewrite bug)"

            # Legacy data preserved in the deprecated tables.
            async with conn.execute(
                "SELECT message_id, document_id, user_id FROM videos_deprecated"
            ) as cur:
                vrows = await cur.fetchall()
            assert vrows == [(1, None, 111)]
            async with conn.execute(
                "SELECT user_id, channel_id, title FROM channels_deprecated"
            ) as cur:
                crows = await cur.fetchall()
            assert crows == [(111, -1001, "pre-migration")]

            # Domain backfill (migration 003): sources + content_items populated.
            async with conn.execute(
                "SELECT account_id, provider, provider_source_id, title, watched " "FROM
sources"
            ) as cur:
                src_rows = await cur.fetchall()
            assert src_rows == [(111, "telegram", "-1001", "pre-migration", 1)]
            async with conn.execute(
                "SELECT account_id, source_id, provider_message_id, document_id, "
                "duration_s, width, height, size_bytes, dedup_key FROM content_items"
            ) as cur:
                ci_rows = await cur.fetchall()
            # dedup_key falls to the Tier-0 branch (no document_id, but the
            # fingerprint fields are all present) ? actually the fingerprint branch
            # wins here since size/duration/h/w are all set.
            assert len(ci_rows) == 1
            ci = ci_rows[0]
            assert ci[0] == 111  # account_id
```

```
            assert ci[2] == "1"  # provider_message_id (cast to TEXT)
            assert ci[3] is None  # document_id
            assert ci[4:8] == (60, 1920, 1080, 1000000)  # dur, w, h, size
            assert ci[8] == "tg:fp:1000000:60:1080:1920"  # fingerprint dedup_key

            # Policy backfill (migration 003): one default policy per account.
            async with conn.execute(
                "SELECT account_id, name, config_json, is_default FROM policies"
            ) as cur:
                pol_rows = await cur.fetchall()
            assert pol_rows == [(111, "default", "{}", 1)]

            # user_version bumped to the highest registered migration.
            async with conn.execute("PRAGMA user_version") as cur:
                assert (await cur.fetchone())[0] == len(db._MIGRATIONS)


    async def test_migration_002_retargeted_fks_allow_inserts_for_new_accounts(
        tmp_path: object,
    ) -> None:
        """Regression guard for PR #33: post-migration, a channel for a brand-new
        account (not present in users_deprecated) must insert cleanly under
        PRAGMA foreign_keys=ON. Catches the SQLite rename-rewrites-FK bug.
        """
        db_path = str(tmp_path) + "/legacy_fk.db"  # type: ignore[operator]
        legacy_schema = """
        CREATE TABLE users (
            id INTEGER PRIMARY KEY, phone TEXT,
            enabled INTEGER NOT NULL DEFAULT 1, created_at TEXT NOT NULL
        );
        CREATE TABLE channels (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            user_id INTEGER NOT NULL REFERENCES users(id),
            channel_id INTEGER NOT NULL, title TEXT, seen_first_at TEXT NOT NULL,
            UNIQUE (user_id, channel_id)
        );
        INSERT INTO users (id, phone, enabled, created_at) VALUES (111, '+1', 1,
'2024-01-01T00:00:00Z');
        """
        async with aiosqlite.connect(db_path) as conn:
            await conn.executescript(legacy_schema)
            await conn.commit()

        await db.init_db(db_path)

        async with aiosqlite.connect(db_path) as conn:
            await conn.execute("PRAGMA foreign_keys=ON")
            # Create a NEW account that exists only in accounts, not users_deprecated.
            await conn.execute(
                "INSERT INTO accounts (id, telegram_id, provider, created_at) "
                "VALUES (222, 222, 'telegram', '2024-01-01T00:00:00Z')"
            )
            # This insert must succeed: channels_deprecated.user_id now references
            # accounts(id), and account 222 exists. Under the bug it would raise FK
            # constraint failed. (Post-migration-003 the table is channels_deprecated.)
            await conn.execute(
                "INSERT INTO channels_deprecated (user_id, channel_id, title, seen_first_at)
"
                "VALUES (222, -2002, 'new-account', '2024-01-01T00:00:00Z')"
            )
            await conn.commit()


    async def test_migration_002_is_idempotent_on_fresh_db(tmp_path: object) -> None:
        """Migration 002 is a no-op on a fresh DB (accounts already exists)."""
```

```
        db_path = str(tmp_path) + "/fresh.db"  # type: ignore[operator]
        await db.init_db(db_path)
        async with aiosqlite.connect(db_path) as conn:
            # Fresh DB has accounts, no users, no users_deprecated.
            assert await db._table_exists(conn, "accounts")
            assert not await db._table_exists(conn, "users")
            assert not await db._table_exists(conn, "users_deprecated")


    async def test_migration_is_idempotent(tmp_path: object) -> None:
        """Running init_db twice must not error (idempotent migration + schema)."""
        db_path = str(tmp_path) + "/idempotent.db"  # type: ignore[operator]
        await db.init_db(db_path)
        # Second run re-applies schema + migrations; must succeed and stay current.
        await db.init_db(db_path)
        async with aiosqlite.connect(db_path) as conn:
            async with conn.execute("PRAGMA user_version") as cur:
                assert (await cur.fetchone())[0] == len(db._MIGRATIONS)
            assert await db._column_exists(conn, "content_items", "document_id")
            assert await db._table_exists(conn, "content_items")
            assert await db._table_exists(conn, "sources")
            assert await db._table_exists(conn, "policies")


    async def test_document_id_unique_index_enforces_dedup(conn: aiosqlite.Connection) ->
None:
        """The partial unique index must reject a second row with the same document_id."""
        import sqlite3

        await _seed_user(conn)
        await db.insert_video(
            conn,
            VideoRecord(user_id=USER_ID, channel_id=CHANNEL_ID, message_id=1,
document_id=999),
        )
        # Same document_id, different (channel, message) ? must violate the index.
        with pytest.raises(sqlite3.IntegrityError, match="UNIQUE"):
            await db.insert_video(
                conn,
                VideoRecord(user_id=USER_ID, channel_id=CHANNEL_ID, message_id=2,
document_id=999),
            )
        # But NULL document_id rows must coexist (partial index).
        await db.insert_video(
            conn,
            VideoRecord(user_id=USER_ID, channel_id=CHANNEL_ID, message_id=3,
document_id=None),
        )
        await db.insert_video(
            conn,
            VideoRecord(user_id=USER_ID, channel_id=CHANNEL_ID, message_id=4,
document_id=None),
        )


    # --- compute_dedup_key (ADR 0011, Option 1A) --------------------------------


    def test_compute_dedup_key_telegram_document_id_wins() -> None:
        """Tier-1 (document_id) is the strongest key; it takes precedence."""
        key = db.compute_dedup_key(
            provider="telegram",
            document_id="12345",
            source_id=1,
            provider_message_id="99",
```

```
            size_bytes=1000,
            duration_s=60,
            height=1080,
            width=1920,
        )
        assert key == "tg:doc:12345"


    def test_compute_dedup_key_telegram_fingerprint_when_no_document_id() -> None:
        """Tier-2 (fingerprint) applies when document_id is absent AND all four
        metadata fields are present (the guard against degenerate tg:fp:0:0:0:0)."""
        key = db.compute_dedup_key(
            provider="telegram",
            size_bytes=1000,
            duration_s=60,
            height=1080,
            width=1920,
        )
        assert key == "tg:fp:1000:60:1080:1920"


    def test_compute_dedup_key_telegram_tier0_when_partial_metadata() -> None:
        """Tier-0 (message id) applies when fingerprint is incomplete ? prevents
        the partial-metadata collision (e.g. missing width ? must NOT use fp branch)."""
        key = db.compute_dedup_key(
            provider="telegram",
            source_id=7,
            provider_message_id="42",
            size_bytes=1000,
            duration_s=60,
            height=1080,
            width=None,  # missing ? fingerprint branch skipped
        )
        assert key == "tg:msg:7:42"


    def test_compute_dedup_key_telegram_tier0_when_all_metadata_missing() -> None:
        """Tier-0 applies when no media metadata at all."""
        key = db.compute_dedup_key(provider="telegram", source_id=7,
provider_message_id="42")
        assert key == "tg:msg:7:42"


    def test_compute_dedup_key_telegram_empty_document_id_falls_through() -> None:
        """An empty-string document_id is treated as absent (falls to next branch)."""
        key = db.compute_dedup_key(
            provider="telegram",
            document_id="",
            source_id=7,
            provider_message_id="42",
        )
        assert key == "tg:msg:7:42"


    def test_compute_dedup_key_telegram_tier0_requires_source_and_message() -> None:
        """The Tier-0 fallback needs source_id + provider_message_id (Telegram
        message_id is per-channel); raising is better than a degenerate key."""
        with pytest.raises(ValueError, match="source_id and provider_message_id"):
            db.compute_dedup_key(provider="telegram")  # no metadata at all


    def test_compute_dedup_key_other_provider_uses_provider_prefix() -> None:
        """Future providers get a provider-prefixed Tier-0 key (no tg: branch)."""
        key = db.compute_dedup_key(
            provider="whatsapp", source_id="wa-group-1", provider_message_id="wa-msg-9"
```

```
        )
        assert key == "whatsapp:msg:wa-group-1:wa-msg-9"


    def test_compute_dedup_key_other_provider_requires_source_and_message() -> None:
        """Non-telegram providers also need source_id + provider_message_id."""
        with pytest.raises(ValueError, match="provider='whatsapp'"):
            db.compute_dedup_key(provider="whatsapp")


    def test_compute_dedup_key_provider_is_case_insensitive() -> None:
        """Provider prefix is lowercased so Telegram/telegram/TELEGRAM collide safely."""
        assert (
            db.compute_dedup_key(provider="TELEGRAM", document_id="5")
            == db.compute_dedup_key(provider="telegram", document_id="5")
            == "tg:doc:5"
        )


    # --- Source / ContentItem / Policy repos (ADR 0011) ------------------------


    async def test_upsert_source_round_trip(conn: aiosqlite.Connection) -> None:
        """Insert then read back a source via the provider-neutral repo."""
        await _seed_user(conn)
        src = await db.upsert_source(
            conn,
            Source(
                account_id=USER_ID,
                provider="telegram",
                provider_source_id="-100555",
                title="Test Channel",
                kind="channel",
                watched=True,
            ),
        )
        assert src.id is not None
        fetched = await db.get_source_by_provider_id(conn, USER_ID, "telegram", "-100555")
        assert fetched is not None
        assert fetched.title == "Test Channel"
        assert fetched.watched is True
        assert fetched.kind == "channel"


    async def test_upsert_source_updates_on_conflict(conn: aiosqlite.Connection) -> None:
        """Re-upserting the same (account, provider, source_id) updates title/watched."""
        await _seed_user(conn)
        await db.upsert_source(
            conn,
            Source(account_id=USER_ID, provider="telegram", provider_source_id="-1",
watched=True),
        )
        updated = await db.upsert_source(
            conn,
            Source(
                account_id=USER_ID,
                provider="telegram",
                provider_source_id="-1",
                title="Renamed",
                watched=False,
            ),
        )
        assert updated.title == "Renamed"
        assert updated.watched is False
        # Still one row.
```

```
        fetched = await db.get_source_by_provider_id(conn, USER_ID, "telegram", "-1")
        assert fetched is not None
        assert fetched.watched is False


    async def test_insert_content_item_round_trip(conn: aiosqlite.Connection) -> None:
        """Insert a content item and read it back via both natural keys."""
        await _seed_user(conn)
        src = await db.upsert_source(
            conn, Source(account_id=USER_ID, provider="telegram", provider_source_id="-1")
        )
        assert src.id is not None
        item = await db.insert_content_item(
            conn,
            ContentItem(
                account_id=USER_ID,
                source_id=src.id,
                provider="telegram",
                provider_message_id="42",
                document_id="999",
                duration_s=60,
                width=1920,
                height=1080,
                size_bytes=1000,
                mime_type="video/mp4",
                dedup_key=db.compute_dedup_key(provider="telegram", document_id="999"),
            ),
        )
        assert item.id is not None
        by_msg = await db.get_content_item_by_message(conn, USER_ID, src.id, "42")
        assert by_msg is not None
        assert by_msg.document_id == "999"
        by_key = await db.get_content_item_by_dedup_key(conn, USER_ID, "tg:doc:999")
        assert by_key is not None
        assert by_key.id == item.id


    async def test_content_item_dedup_key_uniqueness(conn: aiosqlite.Connection) -> None:
        """UNIQUE(account_id, dedup_key) rejects a second item with the same key."""
        import sqlite3

        await _seed_user(conn)
        src = await db.upsert_source(
            conn, Source(account_id=USER_ID, provider="telegram", provider_source_id="-1")
        )
        assert src.id is not None
        await db.insert_content_item(
            conn,
            ContentItem(
                account_id=USER_ID,
                source_id=src.id,
                provider="telegram",
                provider_message_id="1",
                document_id="777",
                dedup_key="tg:doc:777",
            ),
        )
        with pytest.raises(sqlite3.IntegrityError, match="UNIQUE"):
            await db.insert_content_item(
                conn,
                ContentItem(
                    account_id=USER_ID,
                    source_id=src.id,
                    provider="telegram",
                    provider_message_id="2",  # different message, same dedup key
```

```
                    document_id="777",
                    dedup_key="tg:doc:777",
                ),
            )


    async def test_policy_upsert_and_default_lookup(conn: aiosqlite.Connection) -> None:
        """Upsert a default policy and read it back via get_default_policy."""
        await _seed_user(conn)
        policy = await db.upsert_policy(
            conn,
            Policy(
                account_id=USER_ID,
                name="default",
                config_json='{"min_duration_s":5}',
                is_default=True,
            ),
        )
        assert policy.id is not None
        fetched = await db.get_default_policy(conn, USER_ID)
        assert fetched is not None
        assert fetched.config_json == '{"min_duration_s":5}'
        assert fetched.is_default is True


    async def test_filter_config_shim_uses_default_policy(conn: aiosqlite.Connection) ->
None:
        """save_filter_config ? load_filter_config round-trips through policies."""
        await _seed_user(conn)
        custom = FilterConfig(min_duration_s=30, max_duration_s=600, target_height=720)
        await db.save_filter_config(conn, USER_ID, custom)
        loaded = await db.load_filter_config(conn, USER_ID)
        assert loaded.min_duration_s == 30
        assert loaded.target_height == 720
        # The default policy row exists.
        policy = await db.get_default_policy(conn, USER_ID)
        assert policy is not None
        assert policy.is_default is True


    # --- migration 003: legacy data backfill (ADR 0011) ------------------------


    async def test_migration_003_backfills_all_dedup_key_branches(tmp_path: object) -> None:
        """The content_items backfill must exercise all three dedup_key branches
        (document_id / fingerprint / Tier-0 message) and never collapse two rows
        to the same key. Seeds rows hitting each branch on a legacy DB.
        """
        db_path = str(tmp_path) + "/legacy_dedup.db"  # type: ignore[operator]
        legacy_schema = """
        CREATE TABLE users (
            id INTEGER PRIMARY KEY, phone TEXT,
            enabled INTEGER NOT NULL DEFAULT 1, created_at TEXT NOT NULL
        );
        CREATE TABLE channels (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            user_id INTEGER NOT NULL REFERENCES users(id),
            channel_id INTEGER NOT NULL, title TEXT, seen_first_at TEXT NOT NULL,
            UNIQUE (user_id, channel_id)
        );
        CREATE TABLE videos (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            user_id INTEGER NOT NULL REFERENCES users(id),
            channel_id INTEGER NOT NULL, message_id INTEGER NOT NULL,
            document_id INTEGER, duration_s INTEGER, width INTEGER, height INTEGER,
```

```
            size_bytes INTEGER, mime_type TEXT, matched INTEGER NOT NULL DEFAULT 0,
            forwarded INTEGER NOT NULL DEFAULT 0, seen_at TEXT NOT NULL,
            UNIQUE (user_id, channel_id, message_id)
        );
        CREATE TABLE filter_config (
            user_id INTEGER PRIMARY KEY REFERENCES users(id), config_json TEXT NOT NULL
        );
        """
        async with aiosqlite.connect(db_path) as conn:
            await conn.executescript(legacy_schema)
            await conn.execute(
                "INSERT INTO users (id, phone, enabled, created_at) "
                "VALUES (111, '+1', 1, '2024-01-01T00:00:00Z')"
            )
            # Two channels so message_id-only keys don't collide.
            await conn.executemany(
                "INSERT INTO channels (user_id, channel_id, title, seen_first_at) "
                "VALUES (111, ?, ?, '2024-01-01T00:00:00Z')",
                [(-1001, "Channel A"), (-1002, "Channel B")],
            )
            await conn.executemany(
                "INSERT INTO videos (user_id, channel_id, message_id, document_id, "
                "duration_s, width, height, size_bytes, mime_type, matched, forwarded,
seen_at) "
                "VALUES (111, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, '2024-01-01T00:00:00Z')",
                [
                    # Branch 1: document_id set ? tg:doc:
                    (-1001, 1, 5000, 60, 1920, 1080, 1000, "video/mp4", 1, 0),
                    # Branch 2: no document_id, all four fingerprint fields ? tg:fp:
                    (-1001, 2, None, 60, 1920, 1080, 2000, "video/mp4", 0, 0),
                    # Branch 3: no document_id, partial metadata (no width) ? tg:msg:
                    (-1002, 3, None, 60, None, 1080, 3000, "video/mp4", 0, 0),
                    # Branch 3: no document_id, no metadata at all ? tg:msg:
                    (-1002, 4, None, None, None, None, None, None, 0, 0),
                ],
            )
            await conn.execute(
                "INSERT INTO filter_config (user_id, config_json) VALUES (111,
'{\"min_duration_s\":5}')"
            )
            await conn.commit()

        await db.init_db(db_path)

        async with aiosqlite.connect(db_path) as conn:
            # All four videos migrated, each with a distinct dedup_key. The
            # exact source_id in the tg:msg: keys is the surrogate PK of the
            # channel -1002's sources row (inserted second ? id=2); the critical
            # invariant is that all four keys are distinct (no partial-metadata
            # collision collapsed two rows to the same key).
            async with conn.execute("SELECT dedup_key FROM content_items ORDER BY
dedup_key") as cur:
                keys = [r[0] for r in await cur.fetchall()]
            assert len(keys) == 4, f"expected 4 content items, got {len(keys)}"
            assert len(set(keys)) == 4, f"dedup_key collision: {keys}"
            # Branch coverage: one tg:doc:, one tg:fp:, two tg:msg: (different
            # message ids ? distinct keys even with the same source_id).
            prefixes = {k.split(":", 2)[1] for k in keys}
            assert prefixes == {"doc", "fp", "msg"}
            assert sum(1 for k in keys if k.startswith("tg:doc:")) == 1
            assert sum(1 for k in keys if k.startswith("tg:fp:")) == 1
            assert sum(1 for k in keys if k.startswith("tg:msg:")) == 2

            # sources backfilled with watched=1 (ADR 0009 backfill).
            async with conn.execute(
```

```
1203                "SELECT provider_source_id, watched FROM sources ORDER BY
provider_source_id"
1204            ) as cur:
1205                src_rows = await cur.fetchall()
1206            assert src_rows == [("-1001", 1), ("-1002", 1)]
1207
1208            # policy backfilled.
1209            async with conn.execute(
1210                "SELECT name, is_default FROM policies WHERE account_id = 111"
1211            ) as cur:
1212                pol = await cur.fetchone()
1213            assert pol == ("default", 1)
1214
1215
1216    async def test_migration_003_is_idempotent(tmp_path: object) -> None:
1217        """Running migration 003 twice (via init_db) is a no-op on the second run."""
1218        db_path = str(tmp_path) + "/idem003.db"  # type: ignore[operator]
1219        await db.init_db(db_path)
1220        await db.init_db(db_path)  # second run
1221        async with aiosqlite.connect(db_path) as conn:
1222            async with conn.execute("PRAGMA user_version") as cur:
1223                assert (await cur.fetchone())[0] == len(db._MIGRATIONS)
1224            # New tables present, legacy tables absent (fresh DB).
1225            assert await db._table_exists(conn, "sources")
1226            assert await db._table_exists(conn, "content_items")
1227            assert await db._table_exists(conn, "policies")
1228            assert not await db._table_exists(conn, "channels")
1229
```

5. assistant (2026-07-06T06:03:03.603Z)

Now let me read the db.py migration and shim code, and test_delivery.py, to verify the claims.

6. user (2026-07-06T06:03:04.644Z)

```
1       """Tests for the link-message builder and delivery dispatcher.
2
3       The pure formatting (``build_link_text`` and its helpers) is fully unit-tested.
4       The ``deliver`` dispatcher is tested via fakes (FakeClient) since it wraps
5       Telethon calls.
6       """
7
8       from __future__ import annotations
9
10      from unittest.mock import AsyncMock
11
12      import aiosqlite
13      import pytest
14      from telethon.errors import FloodWaitError
15
16      from tg_video_filter import db
17      from tg_video_filter.delivery import build_link_text, deliver
18      from tg_video_filter.models import DeliveryMode, User, VideoRecord
19
20      USER_ID = 1
21      CHANNEL_ID = 100
22      MSG_ID = 7
23
24
25      # --- build_link_text (pure formatting) ------------------------------------
26
27
28      def test_link_text_full_metadata() -> None:
29          text = build_link_text(
30              message_link="https://t.me/c/123/42",
```

```
            duration_s=300,
            height=1080,
            size_bytes=500 * 1024 * 1024,
            channel_title="Movie Channel",
        )
        assert "? 1080p · 5min · 500MB" in text
        assert "#Movie_Channel" in text
        assert "https://t.me/c/123/42" in text


    def test_link_text_no_link() -> None:
        """When message_link is None (e.g. inaccessible channel), the link line is
omitted."""
        text = build_link_text(
            message_link=None,
            duration_s=120,
            height=720,
            size_bytes=50 * 1024 * 1024,
            channel_title="Clips",
        )
        assert "? 720p · 2min · 50MB" in text
        assert "#Clips" in text
        assert "https://t.me" not in text


    def test_link_text_no_channel_title() -> None:
        text = build_link_text(
            message_link="https://t.me/c/1/1",
            duration_s=3600,
            height=2160,
            size_bytes=2 * 1024 * 1024 * 1024,
            channel_title=None,
        )
        assert "? 2160p · 1h · 2.0GB" in text
        assert "https://t.me/c/1/1" in text


    def test_link_text_all_none_metadata() -> None:
        """All metadata fields None -> question marks, link still present."""
        text = build_link_text(
            message_link="https://t.me/c/1/1",
            duration_s=None,
            height=None,
            size_bytes=None,
            channel_title=None,
        )
        assert "? ? · ? · ?" in text
        assert "https://t.me/c/1/1" in text


    @pytest.mark.parametrize(
        ("seconds", "expected"),
        [
            (30, "30s"),
            (59, "59s"),
            (60, "1min"),
            (300, "5min"),
            (3600, "1h"),
            (7200, "2h"),
            (3661, "1h01min"),
        ],
    )
    def test_format_duration(seconds: int, expected: str) -> None:
        from tg_video_filter.delivery import _format_duration

```

```
        assert _format_duration(seconds) == expected


    @pytest.mark.parametrize(
        ("mib", "expected"),
        [
            (1, "1MB"),
            (500, "500MB"),
            (1024, "1.0GB"),
            (2048, "2.0GB"),
        ],
    )
    def test_format_size(mib: int, expected: str) -> None:
        from tg_video_filter.delivery import _format_size

        assert _format_size(mib * 1024 * 1024) == expected


    # --- deliver() dispatcher ------------------------------------------------


    class FakeClient:
        """Tracks forward_messages and send_message calls and their target args."""

        def __init__(self, side_effects: list[BaseException | None] | None = None) -> None:
            self._side_effects = list(side_effects or [])
            self.forward_calls = 0
            self.send_calls = 0
            # Record the first positional arg (the target entity) of each call.
            self.forward_targets: list[object] = []
            self.send_targets: list[object] = []

        async def forward_messages(self, *a, **kw):
            self.forward_calls += 1
            if a:
                self.forward_targets.append(a[0])
            if self._side_effects:
                effect = self._side_effects.pop(0)
                if effect is not None:
                    raise effect

        async def send_message(self, *a, **kw):
            self.send_calls += 1
            if a:
                self.send_targets.append(a[0])
            if self._side_effects:
                effect = self._side_effects.pop(0)
                if effect is not None:
                    raise effect


    @pytest.fixture
    async def db_with_video() -> tuple[aiosqlite.Connection, int]:
        c = await aiosqlite.connect(":memory:")
        await db._apply_schema(c)
        await db.upsert_user(c, User(id=USER_ID))
        inserted = await db.insert_video(
            c, VideoRecord(user_id=USER_ID, channel_id=CHANNEL_ID, message_id=MSG_ID)
        )
        assert inserted.id is not None
        yield c, inserted.id
        await c.close()


    _SENTINEL_MSG = object()
```

```
160
161
162     @pytest.fixture
163     def _no_sleep(monkeypatch: pytest.MonkeyPatch) -> None:
164         monkeypatch.setattr("tg_video_filter.delivery.asyncio.sleep", AsyncMock())
165
166
167     async def test_deliver_forward_mode(db_with_video: tuple[aiosqlite.Connection, int]) ->
None:
168         c, vid = db_with_video
169         client = FakeClient()
170         ok = await deliver(client, _SENTINEL_MSG, c, vid, DeliveryMode.FORWARD,
link_text=None)  # type: ignore[arg-type]
171         assert ok is True
172         assert client.forward_calls == 1
173         assert client.send_calls == 0
174
175
176     async def test_deliver_link_mode(db_with_video: tuple[aiosqlite.Connection, int]) ->
None:
177         c, vid = db_with_video
178         client = FakeClient()
179         ok = await deliver(
180             client,
181             _SENTINEL_MSG,
182             c,
183             vid,
184             DeliveryMode.LINK,
185             link_text="? test link",  # type: ignore[arg-type]
186         )
187         assert ok is True
188         assert client.forward_calls == 0
189         assert client.send_calls == 1
190
191
192     async def test_deliver_both_mode(db_with_video: tuple[aiosqlite.Connection, int]) ->
None:
193         c, vid = db_with_video
194         client = FakeClient()
195         ok = await deliver(
196             client,
197             _SENTINEL_MSG,
198             c,
199             vid,
200             DeliveryMode.BOTH,
201             link_text="? test link",  # type: ignore[arg-type]
202         )
203         assert ok is True
204         assert client.forward_calls == 1
205         assert client.send_calls == 1
206
207
208     async def test_deliver_link_without_text_falls_back_to_forward(
209         db_with_video: tuple[aiosqlite.Connection, int],
210     ) -> None:
211         """LINK mode with no link_text -> falls back to FORWARD (never drops a match)."""
212         c, vid = db_with_video
213         client = FakeClient()
214         ok = await deliver(client, _SENTINEL_MSG, c, vid, DeliveryMode.LINK, link_text=None)
# type: ignore[arg-type]
215         assert ok is True
216         assert client.forward_calls == 1
217         assert client.send_calls == 0
218
219
```

```
220     async def test_deliver_flood_then_retry(
221         db_with_video: tuple[aiosqlite.Connection, int], _no_sleep: None
222     ) -> None:
223         c, vid = db_with_video
224         client = FakeClient(side_effects=[FloodWaitError(request=None, capture=2), None])
225         ok = await deliver(client, _SENTINEL_MSG, c, vid, DeliveryMode.FORWARD,
link_text=None)  # type: ignore[arg-type]
226         assert ok is True
227         assert client.forward_calls == 2  # initial + 1 retry
228
229
230     async def test_deliver_succeeds_without_forwarded_flag_post_adr_0011(
231         db_with_video: tuple[aiosqlite.Connection, int],
232     ) -> None:
233         """ADR 0011: delivery succeeds but ``forwarded`` stays False (no-op shim).
234
235         Post-migration-003 there is no ``forwarded`` column on ``content_items``;
236         per-destination delivery state moves to ``delivery_attempts`` in Track C.
237         For Phase 1 ``mark_forwarded`` is a no-op, so the projected
238         ``VideoRecord.forwarded`` is always False ? but delivery still returns True.
239         """
240         c, vid = db_with_video
241         client = FakeClient()
242         ok = await deliver(client, _SENTINEL_MSG, c, vid, DeliveryMode.LINK,
link_text="test")  # type: ignore[arg-type]
243         assert ok is True
244         row = await db.get_video(c, USER_ID, CHANNEL_ID, MSG_ID)
245         assert row is not None
246         # forwarded is always False post-ADR-0011 (the column is dropped; delivery
247         # state moves to Track C). The match is still recorded via `matched`.
248         assert row.forwarded is False
249
250
251     async def test_both_mode_logs_warning_on_link_failure(
252         db_with_video: tuple[aiosqlite.Connection, int],
253         caplog: pytest.LogCaptureFixture,
254     ) -> None:
255         """When BOTH mode's link send fails, a warning is logged."""
256         c, vid = db_with_video
257         # First call (forward) succeeds; second (send_message) raises.
258         client = FakeClient(side_effects=[None, RuntimeError("link fail")])
259         await deliver(
260             client,
261             _SENTINEL_MSG,
262             c,
263             vid,
264             DeliveryMode.BOTH,
265             link_text="test",  # type: ignore[arg-type]
266         )
267         assert "link_delivery_failed_in_both_mode" in caplog.text
268
269
270     async def test_link_mode_fallback_to_forward_is_logged(
271         db_with_video: tuple[aiosqlite.Connection, int],
272         caplog: pytest.LogCaptureFixture,
273     ) -> None:
274         """LINK mode with no link_text falls back to FORWARD and logs a warning."""
275         c, vid = db_with_video
276         client = FakeClient()
277         await deliver(client, _SENTINEL_MSG, c, vid, DeliveryMode.LINK, link_text=None)  #
type: ignore[arg-type]
278         assert "link_mode_fallback_to_forward" in caplog.text
279
280
281     # --- delivery target (ADR 0005) ---------------------------------------------
```

```


    async def test_deliver_defaults_to_saved_messages(
        db_with_video: tuple[aiosqlite.Connection, int],
    ) -> None:
        """Omitting target uses 'me' (Saved Messages) ? the v1.1 default."""
        c, vid = db_with_video
        client = FakeClient()
        await deliver(client, _SENTINEL_MSG, c, vid, DeliveryMode.FORWARD, link_text=None)
# type: ignore[arg-type]
        assert client.forward_targets == ["me"]


    async def test_deliver_forward_to_channel_username(
        db_with_video: tuple[aiosqlite.Connection, int],
    ) -> None:
        """FORWARD mode routes to an @username channel."""
        c, vid = db_with_video
        client = FakeClient()
        await deliver(
            client,
            _SENTINEL_MSG,  # type: ignore[arg-type]
            c,
            vid,
            DeliveryMode.FORWARD,
            link_text=None,
            target="@my-feed",
        )
        assert client.forward_targets == ["@my-feed"]
        assert client.send_calls == 0


    async def test_deliver_link_to_numeric_chat_id(
        db_with_video: tuple[aiosqlite.Connection, int],
    ) -> None:
        """LINK mode routes to a numeric chat id (private channel)."""
        c, vid = db_with_video
        client = FakeClient()
        await deliver(
            client,
            _SENTINEL_MSG,  # type: ignore[arg-type]
            c,
            vid,
            DeliveryMode.LINK,
            link_text="? test",
            target="-1001234567890",
        )
        assert client.send_targets == ["-1001234567890"]
        assert client.forward_calls == 0


    async def test_deliver_both_to_channel(
        db_with_video: tuple[aiosqlite.Connection, int],
    ) -> None:
        """BOTH mode routes both the forward and the link to the target."""
        c, vid = db_with_video
        client = FakeClient()
        await deliver(
            client,
            _SENTINEL_MSG,  # type: ignore[arg-type]
            c,
            vid,
            DeliveryMode.BOTH,
            link_text="? test",
            target="@curated",
```

```
        )
        assert client.forward_targets == ["@curated"]
        assert client.send_targets == ["@curated"]

```

7. user (2026-07-06T06:03:05.495Z)

```
        """Deprecated shim: upsert the :class:`Source` implied by a legacy Channel.

        ADR 0011: ``channels`` is renamed to ``channels_deprecated``; this function
        reads/writes ``sources`` instead and maps the legacy ``Channel`` shape back.
        New code should call :func:`upsert_source` directly.
        """
        src = await upsert_source(
            conn,
            Source(
                account_id=channel.user_id,
                provider="telegram",
                provider_source_id=str(channel.channel_id),
                title=channel.title,
                kind="channel",
                watched=True,  # back-compat: pre-ADR-0011 channels were all watched
                seen_first_at=channel.seen_first_at,
            ),
        )
        return _source_to_channel(src)


    async def get_channel(conn: aiosqlite.Connection, user_id: int, channel_id: int) ->
Channel | None:
        """Deprecated shim: project the :class:`Source` row back to a legacy Channel."""
        src = await get_source_by_provider_id(conn, user_id, "telegram", str(channel_id))
        return _source_to_channel(src) if src else None


    def _source_to_channel(src: Source) -> Channel:
        """Project a provider-neutral :class:`Source` onto the legacy :class:`Channel`."""
        return Channel(
            id=src.id,
            user_id=src.account_id,
            channel_id=int(src.provider_source_id),
            title=src.title,
            seen_first_at=src.seen_first_at,
        )


    # --- legacy VideoRecord shims (backwards-compatible view over content_items) -


    async def get_video(
        conn: aiosqlite.Connection, user_id: int, channel_id: int, message_id: int
    ) -> VideoRecord | None:
        """Deprecated shim: Tier-0 lookup against ``content_items`` (ADR 0011).

        Resolves the legacy ``(user_id, channel_id, message_id)`` key to a
        :class:`Source` + :class:`ContentItem` and projects the latter onto a
        :class:`VideoRecord`. New code should call :func:`get_content_item_by_message`.
        """
        src = await get_source_by_provider_id(conn, user_id, "telegram", str(channel_id))
        if src is None:
            return None
        item = await get_content_item_by_message(conn, user_id, src.id, str(message_id))
        return _content_item_to_video(item, src, user_id, channel_id) if item else None


```

```
async def get_video_by_document_id(
    conn: aiosqlite.Connection, user_id: int, document_id: int
) -> VideoRecord | None:
    """Deprecated shim: Tier-1 lookup via ``compute_dedup_key`` + ``content_items``."""
    dedup_key = compute_dedup_key(provider="telegram", document_id=str(document_id))
    item = await get_content_item_by_dedup_key(conn, user_id, dedup_key)
    if item is None:
        return None
    src = await get_source_by_id(conn, item.source_id)
    if src is None:
        return None
    return _content_item_to_video(item, src, user_id, int(src.provider_source_id))


async def get_video_by_content_fingerprint(
    conn: aiosqlite.Connection,
    user_id: int,
    size_bytes: int,
    duration_s: int,
    height: int,
    width: int,
) -> VideoRecord | None:
    """Deprecated shim: Tier-2 lookup via ``compute_dedup_key`` + ``content_items``."""
    dedup_key = compute_dedup_key(
        provider="telegram",
        size_bytes=size_bytes,
        duration_s=duration_s,
        height=height,
        width=width,
    )
    item = await get_content_item_by_dedup_key(conn, user_id, dedup_key)
    if item is None:
        return None
    src = await get_source_by_id(conn, item.source_id)
    if src is None:
        return None
    return _content_item_to_video(item, src, user_id, int(src.provider_source_id))


async def get_source_by_id(conn: aiosqlite.Connection, source_id: int) -> Source | None:
    """Resolve a source by its surrogate PK (used by the legacy video shims)."""
    conn.row_factory = aiosqlite.Row
    async with conn.execute("SELECT * FROM sources WHERE id = ?", (source_id,)) as cur:
        row = await cur.fetchone()
    return _row_to_source(row) if row else None


async def insert_video(conn: aiosqlite.Connection, video: VideoRecord) -> VideoRecord:
    """Deprecated shim: insert into ``content_items`` (ADR 0011).

    Resolves the legacy ``channel_id`` to a :class:`Source` row (upserting it
    if absent), computes the provider-neutral ``dedup_key`` via
    :func:`compute_dedup_key`, and inserts a :class:`ContentItem`. The legacy
    ``VideoRecord`` shape is reconstructed from the inserted row.
    """
    src = await upsert_source(
        conn,
        Source(
            account_id=video.user_id,
            provider="telegram",
            provider_source_id=str(video.channel_id),
            kind="channel",
            watched=True,
```

8. assistant (2026-07-06T06:03:08.464Z)

Now let me look at `mark_forwarded`, `get_video_by_content_fingerprint` shim behavior, and the Tier-2 cross-tier concern. Let me read the mark_forwarded implementation and the content_item_to_video projection.

9. user (2026-07-06T06:03:09.496Z)

```
1200            ),
1201        )
1202        assert src.id is not None  # upsert_source populates the surrogate PK
1203        item = ContentItem(
1204            account_id=video.user_id,
1205            source_id=src.id,
1206            provider="telegram",
1207            provider_message_id=str(video.message_id),
1208            content_type="video",
1209            document_id=str(video.document_id) if video.document_id is not None else None,
1210            duration_s=video.duration_s,
1211            width=video.width,
1212            height=video.height,
1213            size_bytes=video.size_bytes,
1214            mime_type=video.mime_type,
1215            dedup_key=compute_dedup_key(
1216                provider="telegram",
1217                document_id=(str(video.document_id) if video.document_id is not None else
None),
1218                source_id=src.id,
1219                provider_message_id=str(video.message_id),
1220                size_bytes=video.size_bytes,
1221                duration_s=video.duration_s,
1222                height=video.height,
1223                width=video.width,
1224            ),
1225            matched=video.matched,
1226            seen_at=video.seen_at,
1227        )
1228        item = await insert_content_item(conn, item)
1229        return _content_item_to_video(item, src, video.user_id, video.channel_id)
1230
1231
1232    async def mark_forwarded(conn: aiosqlite.Connection, video_id: int) -> None:
1233        """Deprecated shim: no-op for the legacy ``forwarded`` flag (ADR 0011).
1234
1235        Per-destination delivery state moves to ``delivery_attempts`` (Track C,
1236        #28). For Phase 1 (before Track C lands) there is no equivalent column on
1237        ``content_items`` ? the ``matched`` flag alone is sufficient because v1
1238        delivery still targets the single ``delivery_target`` and retried delivery
1239        is harmless (forwards are idempotent). This shim is retained so existing
1240        call sites in ``delivery.py`` keep working without a flag day.
1241        """
1242        # content_items has no ``forwarded`` column post-ADR-0011; nothing to write.
1243        # The row's ``matched`` flag (set at insert time) is the surviving signal.
1244        return None
1245
1246
1247    def _content_item_to_video(
1248        item: ContentItem, src: Source, user_id: int, channel_id: int
1249    ) -> VideoRecord:
1250        """Project a :class:`ContentItem` onto the legacy :class:`VideoRecord`.
1251
1252        ``forwarded`` is always ``False`` post-ADR-0011 (the column is dropped;
1253        per-destination delivery state moves to Track C). Existing callers read it
1254        only to decide whether to skip re-delivery; since forwards are idempotent
1255        the ``False`` default is safe for Phase 1.
1256        """
1257        return VideoRecord(
```

```
            id=item.id,
            user_id=user_id,
            channel_id=channel_id,
            message_id=int(item.provider_message_id),
            document_id=int(item.document_id) if item.document_id is not None else None,
            duration_s=item.duration_s,
            width=item.width,
            height=item.height,
            size_bytes=item.size_bytes,
            mime_type=item.mime_type,
            matched=item.matched,
            forwarded=False,
            seen_at=item.seen_at,
        )


    # --- filter config shims (backwards-compatible view over policies) -----------


    async def load_filter_config(conn: aiosqlite.Connection, user_id: int) -> FilterConfig:
        """Deprecated shim: load the account's default :class:`Policy` as FilterConfig.

        ADR 0011 (Option 2A): ``filter_config`` is renamed to
        ``filter_config_deprecated``; this reads ``policies WHERE is_default=1``
        instead. Falls back to :data:`FilterConfig` defaults if no policy exists.
        """
        policy = await get_default_policy(conn, user_id)
        if policy is None:
            return FilterConfig()
        return FilterConfig.model_validate_json(policy.config_json)


    async def save_filter_config(conn: aiosqlite.Connection, user_id: int, cfg:
FilterConfig) -> None:
        """Deprecated shim: upsert the account's default :class:`Policy`."""
        existing = await get_default_policy(conn, user_id)
        await upsert_policy(
            conn,
            Policy(
                id=existing.id if existing is not None else None,
                account_id=user_id,
                name="default",
                config_json=cfg.model_dump_json(),
                is_default=True,
                created_at=(existing.created_at if existing is not None else _now()),
                updated_at=_now(),
            ),
        )


    # --- bulk helpers (used by the manager on startup) --------------------------


    async def list_videos_for_user(  # v2: used by on-demand /search; not called in v1
        conn: aiosqlite.Connection, user_id: int, limit: int = 100
    ) -> list[VideoRecord]:
        """Deprecated shim: list the account's content items projected onto VideoRecord."""
        conn.row_factory = aiosqlite.Row
        async with conn.execute(
            """SELECT ci.*, s.provider_source_id AS src_pid
               FROM content_items ci
               JOIN sources s ON s.id = ci.source_id
               WHERE ci.account_id = ?
               ORDER BY ci.id DESC LIMIT ?""",
            (user_id, limit),
```

```
1322        ) as cur:
1323            rows = await cur.fetchall()
1324        return [
1325            VideoRecord(
1326                id=r["id"],
1327                user_id=user_id,
1328                channel_id=int(r["src_pid"]),
1329                message_id=int(r["provider_message_id"]),
1330                document_id=(int(r["document_id"]) if r["document_id"] is not None else
None),
1331                duration_s=r["duration_s"],
1332                width=r["width"],
1333                height=r["height"],
1334                size_bytes=r["size_bytes"],
1335                mime_type=r["mime_type"],
1336                matched=bool(r["matched"]),
1337                forwarded=False,
1338                seen_at=r["seen_at"],
1339            )
```

10. assistant (2026-07-06T06:03:14.948Z)

Now I understand the shims well. Let me verify the specific concerns raised. First, the key concern: does `get_video_by_content_fingerprint` correctly handle the cross-tier case (an item with a `document_id` whose fingerprint matches a new upload)? The shim computes `tg:fp:...` and looks it up. But an item that had a `document_id` gets `tg:doc:...` as its dedup_key, so its fingerprint is NOT queryable via the fingerprint shim. Let me verify this is the v1 regression and check v1 behavior.

11. user (2026-07-06T06:03:16.916Z)

```
640:async def get_video_by_content_fingerprint(
766:    "get_video_by_content_fingerprint",
```

12. user (2026-07-06T06:03:20.667Z)

```
async def get_video_by_document_id(
    conn: aiosqlite.Connection, user_id: int, document_id: int
) -> VideoRecord | None:
    """Tier-1 content dedup: same Telegram file id ? same video.

    Telegram reuses ``document.id`` when the same file is forwarded across
    channels, so this catches cross-channel duplicates at zero cost.
    """
    conn.row_factory = aiosqlite.Row
    async with conn.execute(
        """SELECT * FROM videos
           WHERE user_id = ? AND document_id = ?""",
        (user_id, document_id),
    ) as cur:
        row = await cur.fetchone()
    return _row_to_video(row) if row else None


async def get_video_by_content_fingerprint(
    conn: aiosqlite.Connection,
    user_id: int,
    size_bytes: int,
    duration_s: int,
    height: int,
    width: int,
) -> VideoRecord | None:
    """Tier-2 content dedup: metadata fingerprint match ? probably same video.
```

```
    Catches re-uploads where Telegram assigns a new ``document.id`` but the
    file bytes are identical. Uses (size_bytes, duration_s, height, width)
    as a composite key ? the odds of two different videos sharing all four
    values are negligible for this use case.
    """
    conn.row_factory = aiosqlite.Row
    async with conn.execute(
        """SELECT * FROM videos
           WHERE user_id = ?
             AND size_bytes = ?
             AND duration_s = ?
             AND height = ?
             AND width = ?
           LIMIT 1""",
        (user_id, size_bytes, duration_s, height, width),
    ) as cur:
        row = await cur.fetchone()
    return _row_to_video(row) if row else None


async def insert_video(conn: aiosqlite.Connection, video: VideoRecord) -> VideoRecord:
    """Insert a new video row. Raises IntegrityError on a dedup-key collision.

    Two dedup constraints protect against duplicates:
      - ``UNIQUE (user_id, channel_id, message_id)`` ? same message instance.
      - ``UNIQUE (user_id, document_id) WHERE document_id IS NOT NULL`` ?
        same file forwarded across channels (content dedup Tier 1).
    """
    cur = await conn.execute(
        """INSERT INTO videos
             (user_id, channel_id, message_id, document_id, duration_s, width, height,
              size_bytes, mime_type, matched, forwarded, seen_at)
           VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)""",
        (
            video.user_id,
            video.channel_id,
            video.message_id,
            video.document_id,
            video.duration_s,
            video.width,
            video.height,
            video.size_bytes,
            video.mime_type,
            int(video.matched),
            int(video.forwarded),
            video.seen_at.isoformat(),
        ),
    )
    await conn.commit()
    return video.model_copy(update={"id": cur.lastrowid})
```

13. assistant (2026-07-06T06:03:28.883Z)

This confirms a real behavioral regression. In v1, `get_video_by_content_fingerprint` queries the raw metadata columns, so a video that has BOTH a `document_id` AND full fingerprint metadata is findable by fingerprint. In the new shim, that same video gets `dedup_key='tg:doc:...'` (document_id branch wins), so a fingerprint lookup computing `tg:fp:...` will MISS it. This is the exact Tier-2 cross-tier scenario the reviewer flagged. Let me write a repro to confirm.

14. user (2026-07-06T06:03:37.338Z)

v2 shim fingerprint hit: None

15. assistant (2026-07-06T06:03:42.448Z)

Confirmed. In v1 this returned the prior video (dedup catches the re-upload); in v2 it returns `None`, so the re-upload is treated as new and re-delivered. This is a real dedup regression, and critically it is NOT covered by any test ? `test_get_video_by_content_fingerprint_hit` (test_db.py:402) seeds a doc-id-less video only. Let me check how the caller uses this to gauge severity.

16. user (2026-07-06T06:03:44.476Z)

```
src/tg_video_filter/backfill.py:171:                    existing = await
db.get_video_by_document_id(conn, user.id, document_id)
src/tg_video_filter/backfill.py:192:                    existing = await
db.get_video_by_content_fingerprint(
src/tg_video_filter/db.py:1371:    "get_video_by_document_id",
src/tg_video_filter/db.py:1372:    "get_video_by_content_fingerprint",
src/tg_video_filter/telethon_client.py:193:            existing = await
db.get_video_by_document_id(conn, user.id, document_id)
src/tg_video_filter/telethon_client.py:214:            existing = await
db.get_video_by_content_fingerprint(
Binary file src/tg_video_filter/__pycache__/backfill.cpython-313.pyc matches
Binary file src/tg_video_filter/__pycache__/db.cpython-313.pyc matches
Binary file src/tg_video_filter/__pycache__/telethon_client.cpython-313.pyc matches
```

17. user (2026-07-06T06:03:46.918Z)

```
185
186             # Tier 0: same (user, channel, message) instance ? already processed.
187             existing = await db.get_video(conn, user.id, channel_id, message_id)
188             if existing is not None:
189                 return
190
191             # Tier 1: same Telegram document.id ? forwarded/cross-posted file.
192             if document_id is not None:
193                 existing = await db.get_video_by_document_id(conn, user.id, document_id)
194                 if existing is not None:
195                     log.info(
196                         "video_dedup_document_id user=%s doc_id=%s channel=%s msg=%s "
197                         "original_channel=%s original_msg=%s",
198                         user.id,
199                         document_id,
200                         channel_id,
201                         message_id,
202                         existing.channel_id,
203                         existing.message_id,
204                     )
205                     return
206
207             # Tier 2: metadata fingerprint ? same file re-uploaded (rare but possible).
208             if (
209                 meta.size_bytes is not None
210                 and meta.duration_s is not None
211                 and meta.height is not None
212                 and meta.width is not None
213             ):
214                 existing = await db.get_video_by_content_fingerprint(
215                     conn,
216                     user.id,
217                     meta.size_bytes,
218                     meta.duration_s,
219                     meta.height,
220                     meta.width,
221                 )
222                 if existing is not None:
223                     log.info(
224                         "video_dedup_fingerprint user=%s size=%s dur=%s h=%s w=%s "
225                         "channel=%s msg=%s original_channel=%s original_msg=%s",
```

```
226                         user.id,
227                         meta.size_bytes,
228                         meta.duration_s,
229                         meta.height,
230                         meta.width,
231                         channel_id,
232                         message_id,
233                         existing.channel_id,
234                         existing.message_id,
```

18. assistant (2026-07-06T06:03:52.257Z)

This confirms the exact real-world scenario the ADR's Tier-2 exists for: original file forwarded once (gets a document_id), then re-uploaded/re-encoded to another channel with a NEW document.id but identical fingerprint. The caller's Tier-1 check misses (new doc id), falls to Tier-2 fingerprint ? which in v2 now misses because the original's dedup_key is `tg:doc:...` not `tg:fp:...`. Result: a duplicate is delivered that v1 would have suppressed. This is a genuine dedup regression uncovered by any test.

Now let me verify the migration test claim and the shim error-branch coverage claim. Let me run coverage on db.py to confirm the uncovered lines.

19. user (2026-07-06T06:04:01.347Z)

```
src\tg_video_filter\db.py                      256      8     40      3    96%   512, 651, 661,
1103-1104, 1147, 1172, 1382
91 passed in 2.50s
```

20. assistant (2026-07-06T06:04:05.258Z)

Confirmed uncovered: 1103-1104 (get_channel None branch), 1147 & 1172 (the `src is None` guards in the fingerprint/doc shims), 512, 651, 661. Let me check line 512 and 651/661.

21. user (2026-07-06T06:04:05.763Z)

```
505         legacy tables to ``*_deprecated`` (one-release rollback net, ADR 0010
506         pattern). Idempotent: a no-op once the legacy tables have been renamed.
507         """
508         # Idempotency gate: if the legacy videos table is gone, this migration has
509         # already run (or the DB is fresh and _SCHEMA's legacy tables were already
510         # dropped by _apply_schema post-003). Nothing to backfill.
511         if not await _table_exists(conn, "videos"):
512             return
513
514         # 1. Backfill sources from channels. ADR 0009 backfill: already-seen
515         #    channels stay watched (watched=1). provider_source_id is the Telegram
516         #    channel_id (cast to TEXT for the provider-neutral column).
517         await conn.execute(
518             """
519             INSERT OR IGNORE INTO sources
520                 (account_id, provider, provider_source_id, title, kind, watched,
```

22. assistant (2026-07-06T06:04:17.213Z)

Now let me confirm the migration test coverage of the fingerprint-null branch. The reviewer's premise was that the migration test seeds only one branch. But I already read `test_migration_003_backfills_all_dedup_key_branches` (test_db.py:1119-1213), which DOES seed all four branches including all-fingerprint-fields-null (row `(-1002, 4, None, None, None, None, None, ...)` at line 1173) and asserts `len(set(keys)) == 4`. So that ADR requirement IS met. The reviewer's premise about `test_migration_adds_document_id` was looking at the wrong test. Let me note the ADR requirement is satisfied ? not a finding.

Now let me assess the shim `int(src.provider_source_id)` concern for whatsapp-style non-numeric source ids and the `_content_item_to_video` `int(item.provider_message_id)`. These are shims

that only handle telegram, so likely fine. Let me confirm the shim error branches are genuinely dead code or reachable.

Line 1147/1172 (`if src is None: return None`) ? this is reachable only if a content_item references a source_id with no sources row (FK violation / orphan). With FK enforcement on, that's near-unreachable. Line 1103-1104 (`get_channel` returning None) is trivially testable. These are low-value coverage gaps.

Let me now confirm the `test_deliver_succeeds_without_forwarded_flag` assertions are meaningful. The test asserts `row.forwarded is False` and `ok is True`. But `forwarded` is ALWAYS False in the projection regardless of whether mark_forwarded did anything ? so the test cannot actually distinguish a working no-op from a broken one. Let me verify the `matched` survival is asserted anywhere in delivery tests.

23. user (2026-07-06T06:04:18.124Z)

```
247:    # state moves to Track C). The match is still recorded via `matched`.
```

24. assistant (2026-07-06T06:04:24.397Z)

Confirmed. In test_delivery.py, `test_deliver_succeeds_without_forwarded_flag_post_adr_0011` only asserts `forwarded is False` ? which is a hardcoded constant in `_content_item_to_video` (db.py:1269), true regardless of anything mark_forwarded does. The comment claims "The match is still recorded via `matched`" but the test never asserts `row.matched`. So the delivery-side test does not meaningfully verify the no-op is safe; it just verifies it doesn't crash. (The db-side test `test_mark_forwarded_is_noop_post_adr_0011` at test_db.py:261 DOES assert `matched is True`, so the invariant is covered there ? this weakens the delivery-test finding to low.)

Now let me double-check the most important gap: is there ANY test covering the cross-tier fingerprint scenario? Let me search both test files.

25. user (2026-07-06T06:04:25.921Z)

```
300:async def test_video_record_content_fingerprint() -> None:
310:    assert v.content_fingerprint == (5_000_000, 120, 1080, 1920)
313:async def test_video_record_content_fingerprint_none_when_missing_field() -> None:
324:    assert v.content_fingerprint is None
402:async def test_get_video_by_content_fingerprint_hit(conn: aiosqlite.Connection) -> None:
416:    found = await db.get_video_by_content_fingerprint(
428:async def test_get_video_by_content_fingerprint_miss(conn: aiosqlite.Connection) -> None:
432:        await db.get_video_by_content_fingerprint(
755:        # dedup_key falls to the Tier-0 branch (no document_id, but the
756:        # fingerprint fields are all present) ? actually the fingerprint branch
764:        assert ci[8] == "tg:fp:1000000:60:1080:1920"  # fingerprint dedup_key
877:def test_compute_dedup_key_telegram_document_id_wins() -> None:
892:def test_compute_dedup_key_telegram_fingerprint_when_no_document_id() -> None:
893:    """Tier-2 (fingerprint) applies when document_id is absent AND all four
906:    """Tier-0 (message id) applies when fingerprint is incomplete ? prevents
915:        width=None,  # missing ? fingerprint branch skipped
926:def test_compute_dedup_key_telegram_empty_document_id_falls_through() -> None:
961:        db.compute_dedup_key(provider="TELEGRAM", document_id="5")
962:        == db.compute_dedup_key(provider="telegram", document_id="5")
1037:            dedup_key=db.compute_dedup_key(provider="telegram", document_id="999"),
1044:    by_key = await db.get_content_item_by_dedup_key(conn, USER_ID, "tg:doc:999")
1066:            dedup_key="tg:doc:777",
1078:                dedup_key="tg:doc:777",
1121:    (document_id / fingerprint / Tier-0 message) and never collapse two rows
1168:                # Branch 2: no document_id, all four fingerprint fields ? tg:fp:
```

26. assistant (2026-07-06T06:04:31.381Z)

Confirmed: `test_get_video_by_content_fingerprint_hit` (line 402) seeds a video with NO document_id, so its dedup_key is `tg:fp:...` and the fingerprint shim finds it. There is no test where the seeded prior video has a document_id and is then queried by fingerprint. The cross-

tier regression is entirely uncovered.

I have enough evidence. Let me finalize findings, ranked by the severity of the bug each missing/weak test fails to catch.

The primary finding is the missing cross-tier test that would have caught a real dedup regression (blocker-level bug in behavior; the test gap itself is what lets it ship). The migration all-branches test requirement IS satisfied (not a finding). The shim None-branch coverage gaps and the weak delivery test are low.

27. user (2026-07-06T06:05:01.547Z)

Structured output provided successfully